

Introduction à l'architecture des ordinateurs

B.U.T. Informatique

Dans le monde, il y a 10 catégories de personnes : celles qui comprennent cette blague, et celles qui ne la comprennent pas

Organisation

Les heures

- 12h de CM/TD ($\approx$ 4h CM, 8h TD)
- 8h de TP
- Venir en TD/TP en ayant relu et compris le cours
- Venir en TD/TP en ayant préparé les exercices
- Finir les TD/TP en dehors des séances

Les contrôles

- 1 contrôle final sur la dernière séance de TD
- 1 petit projet en TPs.

Le projet

- 8h de TPs sur le projet + travail à la maison si nécessaire
- Projet sur les circuit logiques
- Deux possibilités : un processeur ou un jeu de Pong
- Projet formatif :
 - Sujet long et complexe
 - Pas nécessaire de finir le projet pour avoir 20
 - **Vous devez demander de l'aide à l'enseignant**

Objectif du cours

- Histoire et évolution de l'informatique
- Codage de l'information
- Arithmétique et traitement de l'information
- Algèbre de Boole et circuits logiques
- Étude des composants

Objectif du cours

- 1 Théorie : comment l'absence/présence de courant électrique peut servir à représenter des nombres et des lettres ?

Objectif du cours

- ① Théorie : comment l'absence/présence de courant électrique peut servir à représenter des nombres et des lettres ?
- ② Théorie : comment on effectue des calculs avec cette représentation absence/présence

Objectif du cours

- ① Théorie : comment l'absence/présence de courant électrique peut servir à représenter des nombres et des lettres ?
- ② Théorie : comment on effectue des calculs avec cette représentation absence/présence
- ③ Pratique : comment construire un circuit électrique pour faire ces calculs

Objectif du cours

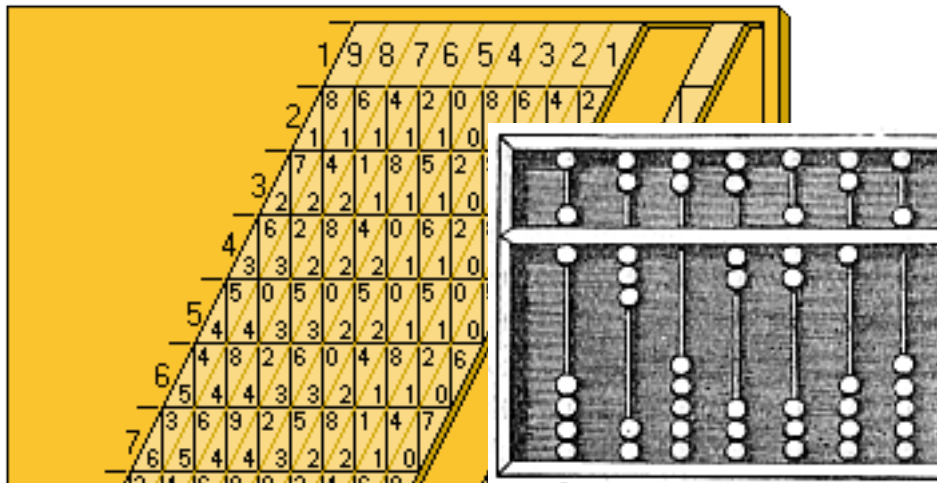
- ① Théorie : comment l'absence/présence de courant électrique peut servir à représenter des nombres et des lettres ?
- ② Théorie : comment on effectue des calculs avec cette représentation absence/présence
- ③ Pratique : comment construire un circuit électrique pour faire ces calculs
- ④ Pratique : comment organiser des composantes autour de ce circuit pour obtenir un ordinateur

Plan

- 1 Historique
- 2 Le codage de l'information
- 3 Arithmétique
- 4 Algèbre de Boole et circuits logiques
- 5 Architecture général

Les premiers outils

Des premier outils d'**aide** au calcul qui remonte à l'antiquité (boulrier, bâton de Neper, règle à calculer, ...)



L'histoire des machines à calculer

- (1645) La Pascaline pour des additions et des soustractions à l'aide de roues dentelées



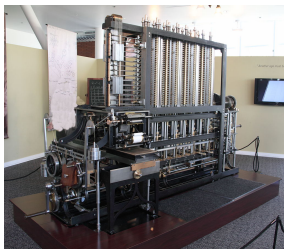
L'histoire des machines à calculer

- (1645) La Pascaline pour des additions et des soustractions à l'aide de roues dentelées
- (1674) Le cylindre cannelé de Leibniz pour des multiplications



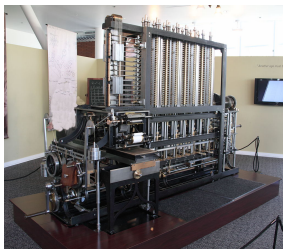
L'histoire des machines à calculer

- (1645) La Pascaline pour des additions et des soustractions à l'aide de roues dentelées
- (1674) Le cylindre cannelé de Leibniz pour des multiplications
- (1822) La machine à différence est proposée par Babbage



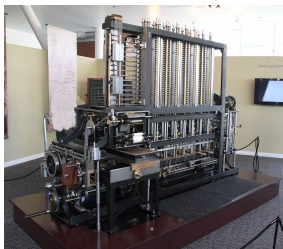
L'histoire des machines à calculer

- (1645) La Pascaline pour des additions et des soustractions à l'aide de roues dentelées
- (1674) Le cylindre cannelé de Leibniz pour des multiplications
- (1822) La machine à différence est proposée par Babbage
- (1834) la machine analytique inventé par Babbage avec une entrée (carte perforée), une sortie, une mémoire et une unité de calcul : le premier ordinateur



L'histoire des machines à calculer

- (1645) La Pascaline pour des additions et des soustractions à l'aide de roues dentelées
- (1674) Le cylindre cannelé de Leibniz pour des multiplications
- (1822) La machine à différence est proposée par Babbage
- (1834) la machine analytique inventé par Babbage avec une entrée (carte perforée), une sortie, une mémoire et une unité de calcul : le premier ordinateur
- (1834) Ada Lovelace écrit le premier programme informatique



Naissance des ordinateurs

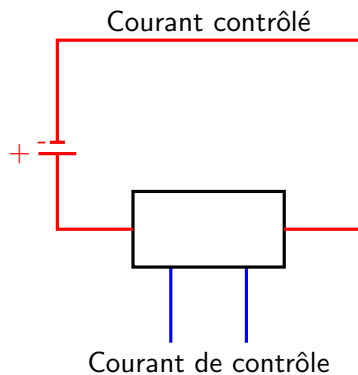
Représenter l'information :

- Représentation binaire, à deux états, et utilisation de la logique de Bool développée au 19ème siècle permettent d'effectuer toutes les opérations logiques et arithmétique.

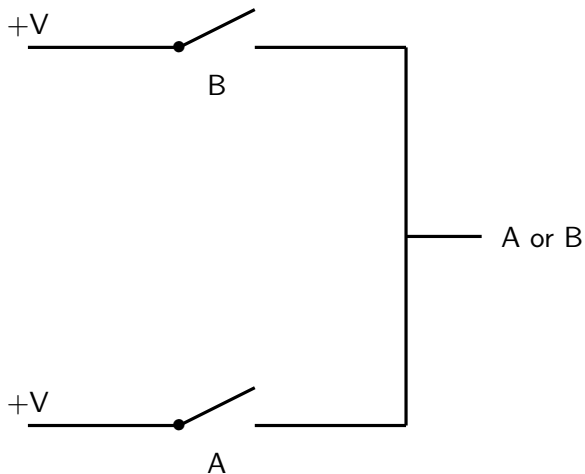
Un bit :

Nom	Valeur
<i>Bit</i> (b)	0 ou 1
<i>octet</i> (o)	00000000 à 11111111
<i>kilobit</i> (kbit ou kb)	10^3 bits
<i>mégabit</i> (Mbit ou Mb)	10^6 bits
<i>gigabit</i> (Gbit ou Gb)	10^9 bits
<i>térambit</i> (Tbit ou Tb)	10^{12} bits

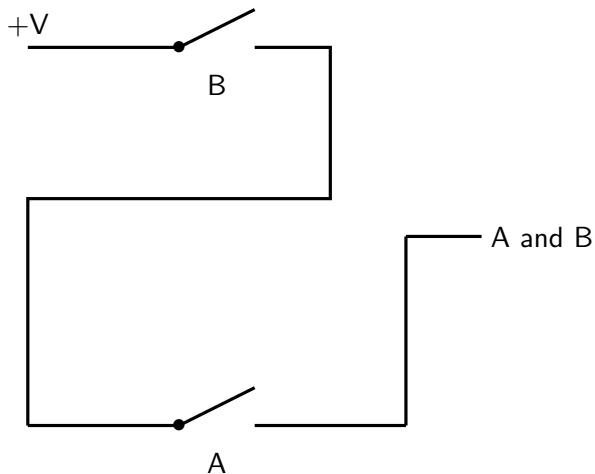
Le principe



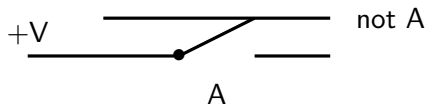
Logique binaire



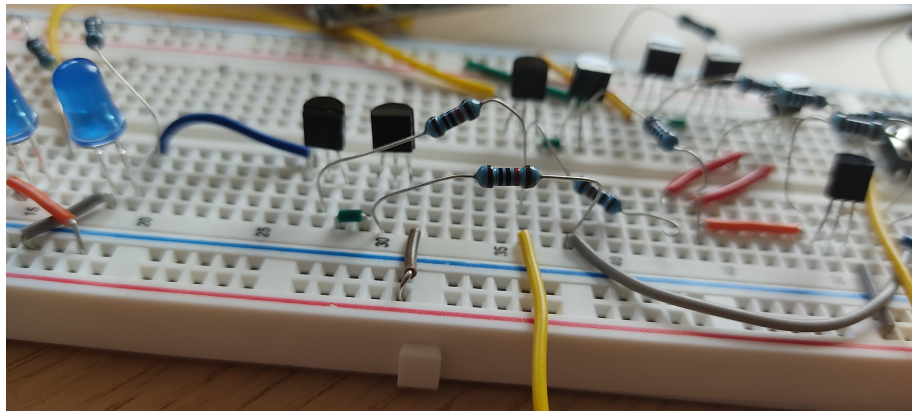
Logique binaire



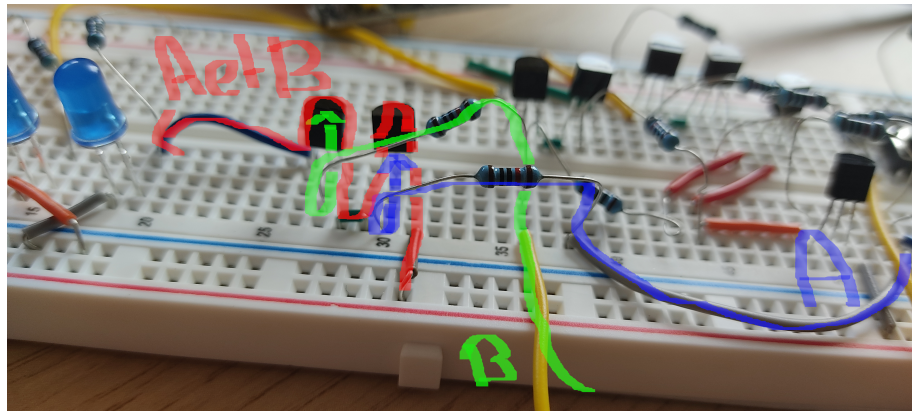
Logique binaire



Illustration



Illustration



Première génération (1936-1956)

Naissance des ordinateurs :

- Atanasoff–Berry Computer (1937), premier ordinateur numérique
- Z3 (1941), 2600 relais électromécaniques pour la première machine programmable
- Colossus (1943), 2400 tubes à vide
- Harvard Mark1 (1944) 765000 composantes pour des multiplications à 23 chiffres en 6s
- ENIAC (1945), 18000 tubes pour des multiplications à 10 chiffres en 3ms
- IBM 701 (1953), commercialisé à 19 exemplaires
- IBM 650 (1953), fabriqué en série à 2000 exemplaire

Deuxième génération (1956-1963)

Deuxième génération :

- Développement des transistors (1947)
- Mémoire à tores magnétique
- Premiers système d'exploitation
- Langage de programmation évoluée
- IBM 1401 (1959), vendu à 10000 exemplaires
- Premier disque dur (1956)

Troisième génération (1963-1971)

Troisième génération :

- Développement des circuits intégrés (1958)
- Miniaturisation des ordinateurs
- Explosion des ventes
- Première compatibilité entre machines

Quatrième génération (1971-)

Quatrième génération :

- Intel 4004 (1971), premier microprocesseur
- Ordinateurs personnels
- Augmentation de la puissance
- Machine virtuelle, base de donnée, parallélisme, . . .

Les ordinateurs de nos jours

Une puissance qui augmente. . .

- Loi de Moore (1965) : la complexité des semi-conducteurs proposés en entrée de gamme double tous les ans à coût constant
- Intel 4004 (1971) : ≈ 2300 transistors, fréquence ≈ 740 kHz
- AMD Zen 2 (2019) : $\approx 4\,000\,000\,000$ transistors, fréquence ≈ 3 GHz

Les ordinateurs de nos jours



Probablement Moore en 1965, Photo d'archive ?

Les ordinateurs de nos jours



Probablement Moore en 1965, Photo d'archive ?

... mais une limite physique qui s'approche...

- Barrière de la chaleur limite la fréquence
- Limite physique sur la taille des composantes (effet quantique)

Les ordinateurs du futur

... mais il y a toujours des améliorations :

- Ordinateur quantique (ou pas)
- Des nouveaux matériaux (portes logiques plus rapides)
- Nouvelles technologies baissant les coûts de production
- Une meilleure “organisation” des processeurs

Plan

- 1 Historique
- 2 Le codage de l'information
- 3 Arithmétique
- 4 Algèbre de Boole et circuits logiques
- 5 Architecture général

Décimal

Représenter un nombre décimal ? Décimal Codé Binaire (BCD)

Code Binaire	Valeur
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9

Décimal : 4254

Décimal

Représenter un nombre décimal ? Décimal Codé Binaire (BCD)

Code Binaire	Valeur
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9

Décimal : 4254

0100 0010 0101 0100

- Codage gourmand
- Opération arithmétique peu efficace

Les bases

Un décimal c'est un nombre en base 10 :

$$4254_{10} = 4 \times 10^3 + 2 \times 10^2 + 5 \times 10^1 + 4 \times 10^0$$

Les bases

Un décimal c'est un nombre en base 10 :

$$4254_{10} = 4 \times 10^3 + 2 \times 10^2 + 5 \times 10^1 + 4 \times 10^0$$

On peut représenter toutes les bases ainsi :

$$15256_7 = 1 \times 7^4 + 5 \times 7^3 + 2 \times 7^2 + 5 \times 7^1 + 6 \times 7^0 = 4254_{10}$$

Les bases

Un décimal c'est un nombre en base 10 :

$$4254_{10} = 4 \times 10^3 + 2 \times 10^2 + 5 \times 10^1 + 4 \times 10^0$$

On peut représenter toutes les bases ainsi :

$$15256_7 = 1 \times 7^4 + 5 \times 7^3 + 2 \times 7^2 + 5 \times 7^1 + 6 \times 7^0 = 4254_{10}$$

Parfait pour les binaires !

$$1000010011110_2 = 4254_{10}$$

Les bases

Un décimal c'est un nombre en base 10 :

$$4254_{10} = 4 \times 10^3 + 2 \times 10^2 + 5 \times 10^1 + 4 \times 10^0$$

On peut représenter toutes les bases ainsi :

$$15256_7 = 1 \times 7^4 + 5 \times 7^3 + 2 \times 7^2 + 5 \times 7^1 + 6 \times 7^0 = 4254_{10}$$

Parfait pour les binaires !

$$1000010011110_2 = 4254_{10}$$

Pas seulement avec des chiffres. Hexadécimal :

$$109E_{16} = 4254_{10}$$

Avec $A = 10_{10}$, $B = 11_{10}$, $C = 12_{10}$, $D = 13_{10}$, $E = 14_{10}$, $F = 15_{10}$. Très utile car un chiffre correspond à 4 binaires.

Décimal	Codage binaire	Décimal	Codage binaire
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

n bits permettent de présenter des nombres de 0 à $2^n - 1$

$$\text{Note : } \begin{array}{r|rr} + & 0 & 1 \\ \hline 0 & 0 & 1 \\ 1 & 1 & 10 \end{array}$$

Passage d'une base à l'autre

Comment exprimer 57_{10} en base 2 ? La division euclidienne !

$$\begin{array}{r|l} 57 & 2 \\ \hline \end{array}$$

Passage d'une base à l'autre

Comment exprimer 57_{10} en base 2 ? La division euclidienne !

$$\begin{array}{r|l} 57 & 2 \\ \hline 1 & 28 \\ & 2 \end{array}$$

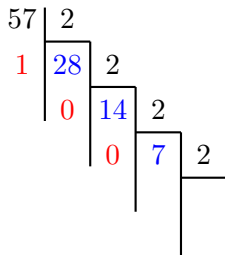
Passage d'une base à l'autre

Comment exprimer 57_{10} en base 2 ? La division euclidienne !

$$\begin{array}{r|l}
 57 & 2 \\
 \hline
 1 & 28 \\
 & 0 \\
 & 14 \\
 & 2
 \end{array}$$

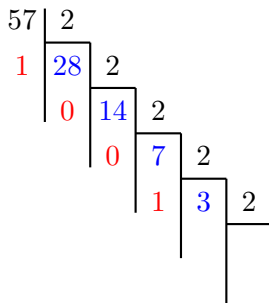
Passage d'une base à l'autre

Comment exprimer 57_{10} en base 2 ? La division euclidienne !



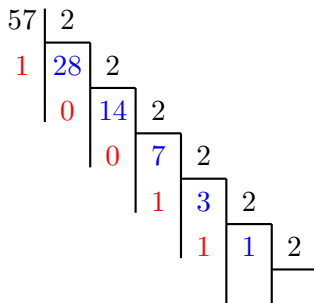
Passage d'une base à l'autre

Comment exprimer 57_{10} en base 2 ? La division euclidienne !



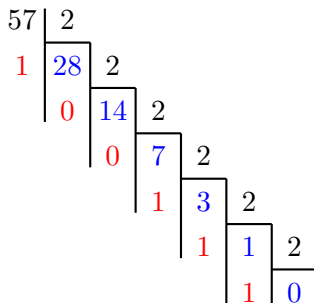
Passage d'une base à l'autre

Comment exprimer 57_{10} en base 2 ? La division euclidienne !



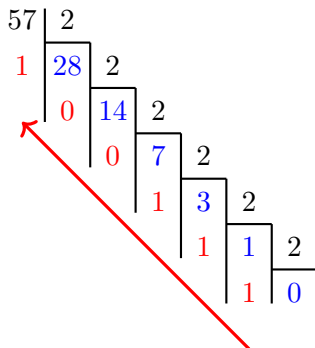
Passage d'une base à l'autre

Comment exprimer 57_{10} en base 2 ? La division euclidienne !



Passage d'une base à l'autre

Comment exprimer 57_{10} en base 2 ? La division euclidienne !



On obtient : $57_{10} = 00111001_2$

Autre codage : Code de Gray

Le code binaire de Gray :

Un seul bit change de valeur pour représenter la valeur suivante ou précédente

Codage décimal	Codage binaire naturel	Codage de Gray
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0						

et

Codage sous forme de Gray	0	0						
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1					

et

Codage sous forme de Gray	0	0						
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0				

et

Codage sous forme de Gray	0	0						
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1			

et

Codage sous forme de Gray	0	0						
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1		

et

Codage sous forme de Gray	0	0						
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1	1	

et

Codage sous forme de Gray	0	0						
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1	1	0

et

Codage sous forme de Gray	0	0						
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1	1	0

et

Codage sous forme de Gray	0	0						
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1	1	0

et

Codage sous forme de Gray	0	0	1					
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1	1	0

et

Codage sous forme de Gray	0	0	1	1				
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1	1	0

et

Codage sous forme de Gray	0	0	1	1	0			
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1	1	0

et

Codage sous forme de Gray	0	0	1	1	0	0		
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1	1	0

et

Codage sous forme de Gray	0	0	1	1	0	0	1	
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1	1	0

et

Codage sous forme de Gray	0	0	1	1	0	0	1	1
Codage binaire naturel	0	0	1	0	0	0	1	0

Transcodage

Utilisation du OU exclusif, dont la table est :

$\oplus$	0	1
0	0	1
1	1	0

Un procédé simple :

Codage sous forme de Gray	0	0	1	1	1	0	0	1
Codage binaire naturel	0	0	1	0	1	1	1	0

et

Codage sous forme de Gray	0	0	1	1	0	0	1	1
Codage binaire naturel	0	0	1	0	0	0	1	0

Questionnement

Oui mais non, nous on veut savoir comment faire avec les entiers positifs ou négatifs !

Questionnement

Oui mais non, nous on veut savoir comment faire avec les entiers positifs ou négatifs !

Les conventions :

- Le bit signé (Signe-magnitude)
- Complément à 1
- Complément à 2
- Biaisée

Note : les exemples de codage seront sur 8 bits et les tableaux sur 4 bits

Le signe-magnitude

Un principe simple :

Le bit de poids fort représente le signe, typiquement 0 pour + et 1 pour −. Utilisé par les premiers ordinateurs IBM commercialisés, utile pour certains processus des années 60 (vidage mémoire) mais peu efficace dans le reste.

Le signe-magnitude

Un principe simple :

Le bit de poids fort représente le signe, typiquement 0 pour + et 1 pour −. Utilisé par les premiers ordinateurs IBM commercialisés, utile pour certains processus des années 60 (vidage mémoire) mais peu efficace dans le reste.

Exemple :

$$+42_{10} = 00101010_2$$

$$-42_{10} = 10101010_2$$

Le signe-magnitude

Un principe simple :

Le bit de poids fort représente le signe, typiquement 0 pour + et 1 pour −. Utilisé par les premiers ordinateurs IBM commercialisés, utile pour certains processus des années 60 (vidage mémoire) mais peu efficace dans le reste.

Exemple :

$$+42_{10} = 00101010_2$$

$$-42_{10} = 10101010_2$$

Problème :

$$+0_{10} = 00000000_2$$

$$-0_{10} = 10000000_2$$

Donc pour comparer deux valeurs, l'ordinateur peut avoir besoin de deux comparaisons ($== +0$ et $== -0$).

Magnitude signée

Chaîne de bits	Non signé	Magnitude signée
0000	0	+0
0001	1	+1
0010	2	+2
0011	3	+3
0100	4	+4
0101	5	+5
0110	6	+6
0111	7	+7
1000	8	-0
1001	9	-1
1010	10	-2
1011	11	-3
1100	12	-4
1101	13	-5
1110	14	-6
1111	15	-7

n bits permettent de présenter des nombres de $-2^{n-1} + 1$ à $2^{n-1} - 1$ ($2^n - 1$ nombres différents)

Complément à 1

Le principe :

Le bit de poids fort représente le signe, 0 pour + et 1 pour -. Un nombre négatif s'obtient en complétant bit par bit sa valeur absolue avec 1.

Conception matérielle plus simple que le signe-magnitude.

Complément à 1

Le principe :

Le bit de poids fort représente le signe, 0 pour + et 1 pour -. Un nombre négatif s'obtient en complétant bit par bit sa valeur absolue avec 1.

Conception matérielle plus simple que le signe-magnitude.

Exemple :

$$+42_{10} = 00101010_2$$

Pour -42_{10} on obtient alors

Complément à 1

Le principe :

Le bit de poids fort représente le signe, 0 pour + et 1 pour -. Un nombre négatif s'obtient en complétant bit par bit sa valeur absolue avec 1.

Conception matérielle plus simple que le signe-magnitude.

Exemple :

$$+42_{10} = 00101010_2$$

$$\begin{array}{r} \text{Pour } -42_{10} \text{ on obtient alors} \\ \begin{array}{r} 11111111 \\ - 00101010 \\ \hline 11010101 \end{array} \end{array}$$

Complément à 1

Le principe :

Le bit de poids fort représente le signe, 0 pour + et 1 pour -. Un nombre négatif s'obtient en complétant bit par bit sa valeur absolue avec 1.

Conception matérielle plus simple que le signe-magnitude.

Exemple :

$$+42_{10} = 00101010_2$$

$$\begin{array}{r} \text{Pour } -42_{10} \text{ on obtient alors} \\ \begin{array}{r} 11111111 \\ - \quad 00101010 \\ \hline = \quad 11010101 \end{array} \end{array}$$

Problème persistant:

$$+0_{10} = 00000000_2$$

$$-0_{10} = 11111111_2$$

Complément à 1

Chaîne de bits	Non signé	Complément à 1
0000	0	+0
0001	1	+1
0010	2	+2
0011	3	+3
0100	4	+4
0101	5	+5
0110	6	+6
0111	7	+7
1000	8	-7
1001	9	-6
1010	10	-5
1011	11	-4
1100	12	-3
1101	13	-2
1110	14	-1
1111	15	-0

n bits permettent de présenter des nombres de $-2^{n-1} + 1$ à $2^{n-1} - 1$ ($2^n - 1$ nombres différents)

Complément à 2

Le principe :

Le bit de poids fort représente le signe, 0 pour + et 1 pour -. Un nombre négatif s'obtient en ajoutant 1 au complément à 1. Conception matérielle la plus simple, et technique la plus utilisée.

Complément à 2

Le principe :

Le bit de poids fort représente le signe, 0 pour + et 1 pour -. Un nombre négatif s'obtient en ajoutant 1 au complément à 1. Conception matérielle la plus simple, et technique la plus utilisée.

Exemple :

$$+42_{10} = 00101010_2$$

$$-42_{10} \text{ en complément à 1 : } 11010101_2$$

$$-42_{10} \text{ en complément à 2 : } 11010101_2 + 1 = 11010110_2$$

$$11010110_2 = -(\text{signe}) \times 2^7 + 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^4 + 1 \times 2^6$$

Complément à 2

Le principe :

Le bit de poids fort représente le signe, 0 pour + et 1 pour -. Un nombre négatif s'obtient en ajoutant 1 au complément à 1. Conception matérielle la plus simple, et technique la plus utilisée.

Exemple :

$$+42_{10} = 00101010_2$$

$$-42_{10} \text{ en complément à 1 : } 11010101_2$$

$$-42_{10} \text{ en complément à 2 : } 11010101_2 + 1 = 11010110_2$$

$$11010110_2 = -(\text{signe}) \times 2^7 + 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^4 + 1 \times 2^6$$

Complément à 2

Le principe :

Le bit de poids fort représente le signe, 0 pour + et 1 pour -. Un nombre négatif s'obtient en ajoutant 1 au complément à 1. Conception matérielle la plus simple, et technique la plus utilisée.

Exemple :

$$+42_{10} = 00101010_2$$

$$-42_{10} \text{ en complément à 1 : } 11010101_2$$

$$-42_{10} \text{ en complément à 2 : } 11010101_2 + 1 = 11010110_2$$

$$11010110_2 = -(\text{signe}) \times 2^7 + 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^4 + 1 \times 2^6$$

Problème persistant ?

Non. Une seule représentation de 0 (00000000_2).

Complément à 2

Chaîne de bits	Non signé	Complément à 2
0000	0	+0
0001	1	+1
0010	2	+2
0011	3	+3
0100	4	+4
0101	5	+5
0110	6	+6
0111	7	+7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

n bits permettent de présenter des nombres de -2^{n-1} à $2^{n-1} - 1$
 (2^n nombres différents)

Biaisé

Le principe :

Aucun bit de signe, on ajoute une valeur (le biais) à tous les nombres que l'on veut représenter, de telle sorte qu'ils soient tous non-négatifs. Utilisé pour les exposants des flottants.

Biaisé

Le principe :

Aucun bit de signe, on ajoute une valeur (le biais) à tous les nombres que l'on veut représenter, de telle sorte qu'ils soient tous non-négatifs. Utilisé pour les exposants des flottants.

Exemple :

Codage 8 bits, avec un biais de 127

-42_{10} est codé $-42 + 127 = 85 = 01010101_2$

$+42_{10}$ est codé $+42 + 127 = 169 = 10101001_2$

Biaisée

Chaîne de bits	Non signé	Biaisée (biais de 7)
0000	0	-7
0001	1	-6
0010	2	-5
0011	3	-4
0100	4	-3
0101	5	-2
0110	6	-1
0111	7	0
1000	8	1
1001	9	2
1010	10	3
1011	11	4
1100	12	5
1101	13	6
1110	14	7
1111	15	8

n bits permettent de présenter des nombres de $-2^{n-1} + 1$ à 2^{n-1}
 (2^n nombres différents)

Youpi ! On est maintenant (presque) des experts en représentation de nombres entiers, négatifs ou positifs !

Youpi ! On est maintenant (presque) des experts en représentation de nombres entiers, négatifs ou positifs !

Mais on peut aussi représenter les caractères, mots, phrases et textes !

Les caractères

Représenter un caractère :

Un caractère (imprimable ou de contrôle) est représenté en binaire, en associant chaque caractère à un nombre binaire

Les caractères

Représenter un caractère :

Un caractère (imprimable ou de contrôle) est représenté en binaire, en associant chaque caractère à un nombre binaire

Plusieurs normes :

- ASCII (American Standard Code for Information Interchange). Représentation sur 7 bits. ASCII étendu sur 8 bits mais pas de normalisation.
- EBCDIC (Extended Binary Coded Decimal Interchange Code). Représentation sur 8 bits . A l'époque beaucoup utilisée par IBM
- ISO/CEI 8859. Représentation sur 8 bits. Divisée en parties (des normes pour différentes langues).
- Unicode (UTF-8, UTF-16, UTF-32), qui peut utiliser 32 bits pour coder 1 caractère, $\approx$ beaucoup de caractères

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Et maintenant, les nombres réels

Principale difficulté :

Impossible de représenter tous les nombres que l'on souhaite, l'ordinateur à une précision limitée.

Un choix :

Quels nombres représentés ?

La virgule fixe

La virgule fixe :

Un premier bit pour le signe, un ensemble de bits pour la partie entière et un ensemble de bits pour la partie réel

La virgule fixe

La virgule fixe :

Un premier bit pour le signe, un ensemble de bits pour la partie entière et un ensemble de bits pour la partie réel

Le changement de base :

$$0000.0101_2 = 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 0.3125_{10}$$

$$0.375_{10} = ???_2$$

La virgule fixe

La virgule fixe :

Un premier bit pour le signe, un ensemble de bits pour la partie entière et un ensemble de bits pour la partie réel

Le changement de base :

$$0000.0101_2 = 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 0.3125_{10}$$

$$0.375_{10} = ???_2$$

$$0.375 \times 2 = 0 + 0.750$$

La virgule fixe

La virgule fixe :

Un premier bit pour le signe, un ensemble de bits pour la partie entière et un ensemble de bits pour la partie réel

Le changement de base :

$$0000.0101_2 = 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 0.3125_{10}$$

$$0.375_{10} = ???_2$$

$$0.375 \times 2 = 0 + 0.750$$

$$0.750 \times 2 = 1 + 0.5$$

La virgule fixe

La virgule fixe :

Un premier bit pour le signe, un ensemble de bits pour la partie entière et un ensemble de bits pour la partie réel

Le changement de base :

$$0000.0101_2 = 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 0.3125_{10}$$

$$0.375_{10} = 0000.0110_2$$

$$0.375 \times 2 = 0 + 0.750$$

$$0.750 \times 2 = 1 + 0.5$$

$$0.5 \times 2 = 1 + 0$$

La virgule fixe

La virgule fixe :

Un premier bit pour le signe, un ensemble de bits pour la partie entière et un ensemble de bits pour la partie réel

Le changement de base :

$$0000.0101_2 = 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 0.3125_{10}$$

$$0.375_{10} = 0000.0110_2$$

$$0.375 \quad \times 2 \quad = 0 + 0.750$$

$$0.750 \quad \times 2 \quad = 1 + 0.5$$

$$0.5 \quad \times 2 \quad = 1 + 0$$

Précision de 2^{-n} avec n le nombre de bits pour la partie fractionnaire (exemple avec 4 bits, on ne peut représenter des parties fractionnaires multiples de 2^{-4} soit 0.0625)

La virgule flottante

La virgule flottante (norme IEEE 754)

Utilisation de la notation scientifique :

$$(-1)^s \times m2^E$$

- Un bit de signe s
- Des bits pour un exposant E
- Des bits pour un mantisse m .

La virgule flottante

La virgule flottante (norme IEEE 754)

Utilisation de la notation scientifique :

$$(-1)^s \times m2^E$$

- Un bit de signe s
- Des bits pour un exposant E
- Des bits pour un mantisse m .

Exemple :

$$\begin{aligned}100110.10 &= 1.0011010 \times 2^5 \\0.0101101 &= 1.0110100 \times 2^{-2}\end{aligned}$$

La virgule flottante

La virgule flottante (norme IEEE 754)

Utilisation de la notation scientifique :

$$(-1)^s \times m2^E$$

- Un bit de signe s
- Des bits pour un exposant E
- Des bits pour un mantisse m .

Exemple :

$$\begin{aligned}100110.10 &= 1.0011010 \times 2^5 \\0.0101101 &= 1.0110100 \times 2^{-2}\end{aligned}$$

Remarque :

Représentée sous la forme $m = 1.f$ donc 1 n'est pas codé (*hidden bit*).
L'exposant est codé avec le biais.

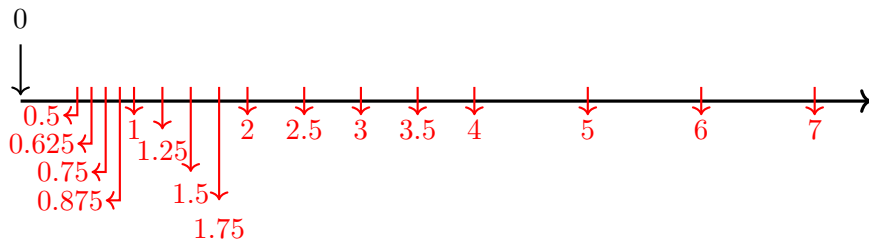
s	E	f
0	10101110	10110010111100100010001

Type	#bits s	#bits E	#bits f
Simple (<i>float</i>)	1	8	23
Double (<i>double</i>)	1	11	52
Long double (<i>long double</i>)	1	15	64

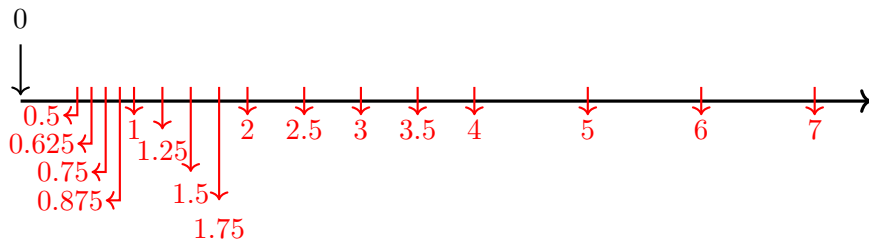
Exemple du format (1,2,2)

Code binaire	Notation Scientifique	Valeur base 10
0 00 00	1.00×2^{-1}	0.5
0 00 01	1.01×2^{-1}	0.625
0 00 10	1.10×2^{-1}	0.75
0 00 11	1.11×2^{-1}	0.875
0 01 00	1.00×2^0	1
0 01 01	1.01×2^0	1.25
0 01 10	1.10×2^0	1.5
0 01 11	1.11×2^0	1.75
0 10 00	1.00×2^1	2
0 10 01	1.01×2^1	2.5
0 10 10	1.10×2^1	3
0 10 11	1.11×2^1	3.5
0 11 00	1.00×2^2	4
0 11 01	1.01×2^2	5
0 11 10	1.10×2^2	6
0 11 11	1.11×2^2	7

Exemple du format (1,2,2)

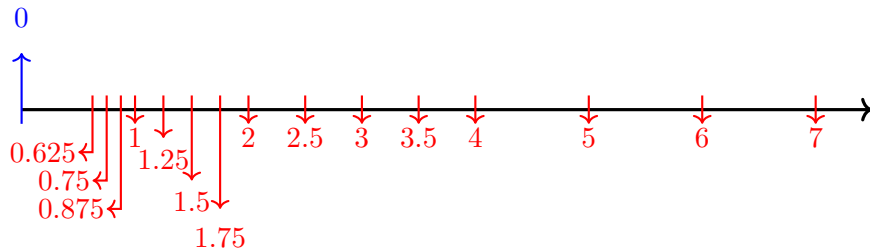


Exemple du format (1,2,2)



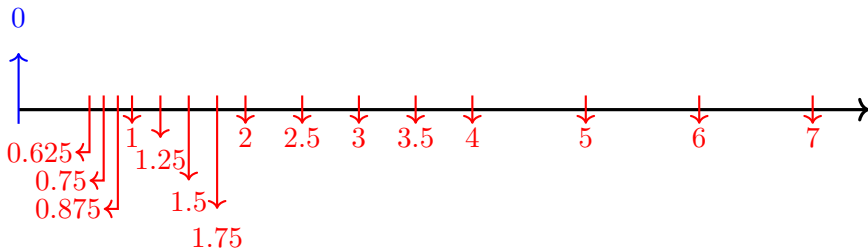
- Pas de représentation de 0

Exemple du format (1,2,2)



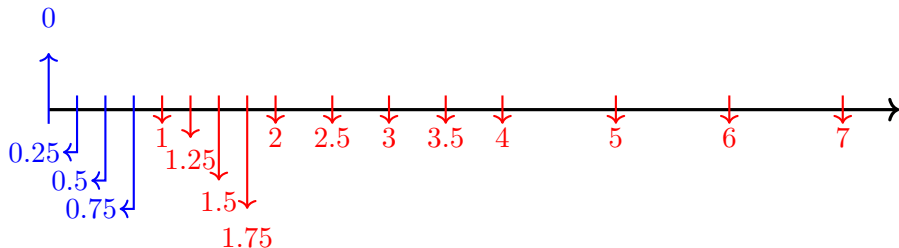
- Pas de représentation de 0 / Réserver 0 00 00 et 1 00 00 (perte de ± 0.5)

Exemple du format (1,2,2)



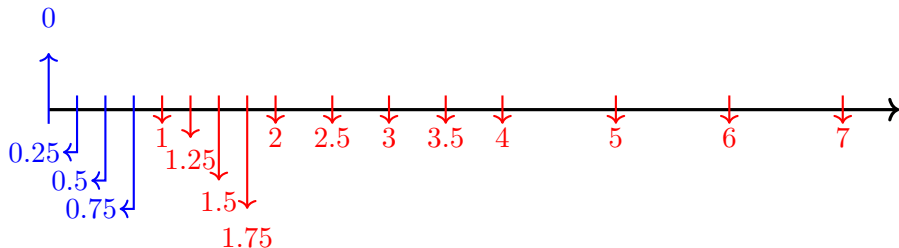
- Pas de représentation de 0 / Réserver 0 00 00 et 1 00 00 (perte de ± 0.5)
- Trou autour de 0

Exemple du format (1,2,2)



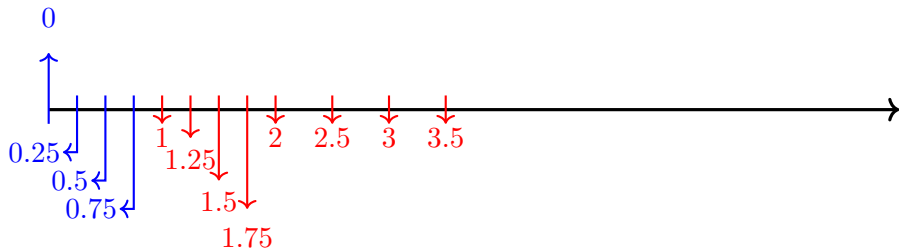
- Pas de représentation de 0 / Réserver 0 00 00 et 1 00 00 (perte de ± 0.5)
- Trou autour de 0 / Réserver $e = 0$ pour les nombres *dénormalisés* (plus de *hidden bit*)

Exemple du format (1,2,2)



- Pas de représentation de 0 / Réserver 0 00 00 et 1 00 00 (perte de ± 0.5)
- Trou autour de 0 / Réserver $e = 0$ pour les nombres *dénormalisés* (plus de *hidden bit*)
- Représentation de l'infini et des NaN (Not a Number : $0/0, \infty/\infty, \infty - \infty, 0 * \infty$)

Exemple du format (1,2,2)



- Pas de représentation de 0 / Réserver 0 00 00 et 1 00 00 (perte de ± 0.5)
- Trou autour de 0 / Réserver $e = 0$ pour les nombres *dénormalisés* (plus de *hidden bit*)
- Représentation de l'infini et des NaN (Not a Number : $0/0, \infty/\infty, \infty - \infty, 0 * \infty$) / Réserver $e = 3$ ($f \neq 0$ alors NaN, $f = 0$ alors $\pm\infty$)

Exemple du format (1,2,2)

Code binaire	Notation Scientifique	Valeur base 10
0 00 00	0.00	0
0 00 01	0.1×2^{-1}	0.25
0 00 10	1.0×2^{-1}	0.5
0 00 11	1.1×2^{-1}	0.75
0 01 00	1.00×2^0	1
0 01 01	1.01×2^0	1.25
0 01 10	1.10×2^0	1.5
0 01 11	1.11×2^0	1.75
0 10 00	1.00×2^1	2
0 10 01	1.01×2^1	2.5
0 10 10	1.10×2^1	3
0 10 11	1.11×2^1	3.5
0 11 00	$+\infty$	$+\infty$
0 11 01	NaN	NaN
0 11 10	NaN	NaN
0 11 11	NaN	NaN

Les arrondis

Un problème :

Lorsque l'on effectue des opérations entre nombres, on peut tomber sur un nombre pas représentable en binaire (ex : $1/3$). Il y a donc une approximation.

Les arrondis

Un problème :

Lorsque l'on effectue des opérations entre nombres, on peut tomber sur un nombre pas représentable en binaire (ex : $1/3$). Il y a donc une approximation.

IEEE 754 :

4 modes d'arrondi :

- $\lfloor x \rfloor$ arrondi vers $-\infty$
- $\lceil x \rceil$ arrondi vers $+\infty$
- Arrondi vers 0 ($\lfloor x \rfloor$ si positif, $\lceil x \rceil$ si négatif)
- Arrondi vers la valeur la plus proche (par défaut). Si à distance égale, on choisit celui avec la mantisse finissant par 0

Plan

- 1 Historique
- 2 Le codage de l'information
- 3 Arithmétique**
- 4 Algèbre de Boole et circuits logiques
- 5 Architecture général

Addition binaire entière non signée

Rappel :

$$\begin{array}{r|rr}
 + & 0 & 1 \\
 \hline
 0 & 0 & 1 \\
 1 & 1 & 10
 \end{array}$$

L'addition reste simple :

$$\begin{array}{rcccccccc}
 & & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & (50_{10}) \\
 + & & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & (119_{10}) \\
 \text{Retenue} & & & & & & & & & & \\
 \hline
 = & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & & (169)_{10}
 \end{array}$$

Addition binaire entière non signée

Rappel :

$$\begin{array}{r|rr}
 + & 0 & 1 \\
 \hline
 0 & 0 & 1 \\
 1 & 1 & 10
 \end{array}$$

L'addition reste simple :

$$\begin{array}{rcccccccccl}
 & & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & (50_{10}) \\
 + & & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & (119_{10}) \\
 \text{Retenue} & & & & & & & & & & \\
 \hline
 = & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & & (169)_{10}
 \end{array}$$

Addition binaire entière non signée

Rappel :

$$\begin{array}{r|rr}
 + & 0 & 1 \\
 \hline
 0 & 0 & 1 \\
 1 & 1 & 10
 \end{array}$$

L'addition reste simple :

$$\begin{array}{rcccccccccc}
 & & 0 & 0 & 1 & 1 & 0 & 0 & \textcolor{red}{1} & 0 & (50_{10}) \\
 + & & 0 & 1 & 1 & 1 & 0 & 1 & \textcolor{red}{1} & 1 & (119_{10}) \\
 \text{Retenue} & & & & & & & \textcolor{blue}{1} & & & \\
 \hline
 = & 1 & 0 & 1 & 0 & 1 & 0 & \textcolor{blue}{0} & 1 & & (169)_{10}
 \end{array}$$

Addition binaire entière non signée

Rappel :

$$\begin{array}{r|rr}
 + & 0 & 1 \\
 \hline
 0 & 0 & 1 \\
 1 & 1 & 10
 \end{array}$$

L'addition reste simple :

$$\begin{array}{rcccccccccl}
 & & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & (50_{10}) \\
 + & & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & (119_{10}) \\
 \text{Retenue} & & & & & & 1 & 1 & & & \\
 \hline
 = & & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & (169)_{10}
 \end{array}$$

Addition binaire entière non signée

Rappel :

$$\begin{array}{r|rr}
 + & 0 & 1 \\
 \hline
 0 & 0 & 1 \\
 1 & 1 & 10
 \end{array}$$

L'addition reste simple :

$$\begin{array}{rcccccccc}
 & & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & (50_{10}) \\
 + & & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & (119_{10}) \\
 \text{Retenue} & & & & & & 1 & & & & \\
 \hline
 = & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & & (169)_{10}
 \end{array}$$

Addition binaire entière non signée

Rappel :

$$\begin{array}{r|rr}
 + & 0 & 1 \\
 \hline
 0 & 0 & 1 \\
 1 & 1 & 10
 \end{array}$$

L'addition reste simple :

$$\begin{array}{rcccccccccl}
 & & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & (50_{10}) \\
 + & & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & (119_{10}) \\
 \text{Retenue} & & & & & 1 & & & & & \\
 \hline
 = & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & & (169)_{10}
 \end{array}$$

Addition binaire entière non signée

Rappel :

$$\begin{array}{r|rr}
 + & 0 & 1 \\
 \hline
 0 & 0 & 1 \\
 1 & 1 & 10
 \end{array}$$

L'addition reste simple :

$$\begin{array}{rcccccccccl}
 & & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & (50_{10}) \\
 + & & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & (119_{10}) \\
 \text{Retenue} & & & 1 & 1 & & & & & & \\
 \hline
 = & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & & (169)_{10}
 \end{array}$$

Addition binaire entière non signée

Rappel :

$$\begin{array}{r|rr}
 + & 0 & 1 \\
 \hline
 0 & 0 & 1 \\
 1 & 1 & 10
 \end{array}$$

L'addition reste simple :

$$\begin{array}{rcccccccccl}
 & & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & (50_{10}) \\
 + & & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & (119_{10}) \\
 \text{Retenue} & & 1 & 1 & & & & & & & \\
 \hline
 = & & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & (169)_{10}
 \end{array}$$

Addition binaire entière non signée

Rappel :

$$\begin{array}{r|rr}
 + & 0 & 1 \\
 \hline
 0 & 0 & 1 \\
 1 & 1 & 10
 \end{array}$$

L'addition reste simple :

$$\begin{array}{rcccccccccl}
 & & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & (50_{10}) \\
 + & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & (119_{10}) \\
 \text{Retenue} & 1 & & & & & & & & & \\
 \hline
 = & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & & (169)_{10}
 \end{array}$$

Soustraction binaire entière non signée

Rappel :

- $0-0 = 0$
- $0-1 = 1$ et on “prend” 1 à gauche
- $1-0 = 1$
- $1-1 = 0$

La soustraction aussi est simple :

	0	1	1	1	0	1	1	0	(118_{10})
—	0	0	1	1	0	0	1	1	(51_{10})
-Retenue									
=	0	1	0	0	0	0	1	1	$(67)_{10}$

Soustraction binaire entière non signée

Rappel :

- $0-0 = 0$
- $0-1 = 1$ et on “prend” 1 à gauche
- $1-0 = 1$
- $1-1 = 0$

La soustraction aussi est simple :

	0	1	1	1	0	1	1	0	(118) ₁₀
—	0	0	1	1	0	0	1	1	(51) ₁₀
-Retenue							1		
=	0	1	0	0	0	0	1	1	(67) ₁₀

Soustraction binaire entière non signée

Rappel :

- $0-0 = 0$
- $0-1 = 1$ et on “prend” 1 à gauche
- $1-0 = 1$
- $1-1 = 0$

La soustraction aussi est simple :

	0	1	1	1	0	1	1	0	(118_{10})
—	0	0	1	1	0	0	1	1	(51_{10})
-Retenue						1	1		
=	0	1	0	0	0	0	1	1	$(67)_{10}$

Soustraction binaire entière non signée

Rappel :

- $0-0 = 0$
- $0-1 = 1$ et on “prend” 1 à gauche
- $1-0 = 1$
- $1-1 = 0$

La soustraction aussi est simple :

	0	1	1	1	0	1	1	0	(118_{10})
—	0	0	1	1	0	0	1	1	(51_{10})
-Retenue						1			
=	0	1	0	0	0	0	1	1	$(67)_{10}$

Soustraction binaire entière non signée

Rappel :

- $0-0 = 0$
- $0-1 = 1$ et on “prend” 1 à gauche
- $1-0 = 1$
- $1-1 = 0$

La soustraction aussi est simple :

	0	1	1	1	0	1	1	0	(118_{10})
—	0	0	1	1	0	0	1	1	(51_{10})
-Retenue									
=	0	1	0	0	0	0	1	1	$(67)_{10}$

Soustraction binaire entière non signée

Rappel :

- $0-0 = 0$
- $0-1 = 1$ et on “prend” 1 à gauche
- $1-0 = 1$
- $1-1 = 0$

La soustraction aussi est simple :

	0	1	1	1	0	1	1	0	(118_{10})
—	0	0	1	1	0	0	1	1	(51_{10})
-Retenue									
=	0	1	0	0	0	0	1	1	$(67)_{10}$

Soustraction binaire entière non signée

Rappel :

- $0-0 = 0$
- $0-1 = 1$ et on “prend” 1 à gauche
- $1-0 = 1$
- $1-1 = 0$

La soustraction aussi est simple :

	0	1	1	1	0	1	1	0	(118_{10})
—	0	0	1	1	0	0	1	1	(51_{10})
-Retenue									
=	0	1	0	0	0	0	1	1	$(67)_{10}$

Soustraction binaire entière non signée

Rappel :

- $0-0 = 0$
- $0-1 = 1$ et on “prend” 1 à gauche
- $1-0 = 1$
- $1-1 = 0$

La soustraction aussi est simple :

	0	1	1	1	0	1	1	0	(118_{10})
—	0	0	1	1	0	0	1	1	(51_{10})
-Retenue									
=	0	1	0	0	0	0	1	1	$(67)_{10}$

Soustraction binaire entière non signée

Rappel :

- $0-0 = 0$
- $0-1 = 1$ et on “prend” 1 à gauche
- $1-0 = 1$
- $1-1 = 0$

La soustraction aussi est simple :

$$\begin{array}{r}
 \\
 - \\
 \hline
 \text{-Retenue} \\
 =
 \end{array}
 \begin{array}{cccccccc}
 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & (118)_{10} \\
 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & (51)_{10} \\
 \hline
 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & (67)_{10}
 \end{array}$$

Addition binaire entière signée

Pour additionner :

Pour additionner des entiers signés, on utilise le complément à 2 où le processus est le plus simple et unifié. La soustraction c'est l'addition de (-) la valeur.

L'addition est sont simple :

$$\begin{array}{rcccccccccl}
& 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & (-114_{10}) \\
+ & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & (65_{10}) \\
\hline
= & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & (-49)_{10}
\end{array}$$

Addition binaire entière signée

Pour additionner :

Pour additionner des entiers signés, on utilise le complément à 2 où le processus est le plus simple et unifié. La soustraction c'est l'addition de (-) la valeur.

L'addition est simple :

$$\begin{array}{rcccccccccl}
 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & (-114_{10}) \\
 + & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & (65_{10}) \\
 \hline
 = & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & (-49)_{10}
 \end{array}$$

Addition binaire entière signée

Pour additionner :

Pour additionner des entiers signés, on utilise le complément à 2 où le processus est le plus simple et unifié. La soustraction c'est l'addition de (-) la valeur.

L'addition est simple :

$$\begin{array}{rcccccccccc}
 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & (-114_{10}) \\
 + & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & (65_{10}) \\
 \hline
 = & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & (-49)_{10}
 \end{array}$$

Addition binaire entière signée

Pour additionner :

Pour additionner des entiers signés, on utilise le complément à 2 où le processus est le plus simple et unifié. La soustraction c'est l'addition de (-) la valeur.

L'addition est simple :

$$\begin{array}{rrrrrrrrrr}
& 1 & 0 & 0 & 0 & 1 & \textcolor{red}{1} & 1 & 0 & (-114_{10}) \\
+ & 0 & 1 & 0 & 0 & 0 & \textcolor{red}{0} & 0 & 1 & (65_{10}) \\
\hline
= & 1 & 1 & 0 & 0 & 1 & \textcolor{blue}{1} & 1 & 1 & (-49)_{10}
\end{array}$$

Addition binaire entière signée

Pour additionner :

Pour additionner des entiers signés, on utilise le complément à 2 où le processus est le plus simple et unifié. La soustraction c'est l'addition de (-) la valeur.

L'addition est simple :

$$\begin{array}{rcccccccccl}
 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & (-114_{10}) \\
 + & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & (65_{10}) \\
 \hline
 = & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & (-49)_{10}
 \end{array}$$

Addition binaire entière signée

Pour additionner :

Pour additionner des entiers signés, on utilise le complément à 2 où le processus est le plus simple et unifié. La soustraction c'est l'addition de (-) la valeur.

L'addition est sont simple :

$$\begin{array}{rcccccccc}
& 1 & 0 & 0 & \textcolor{red}{0} & 1 & 1 & 1 & 0 & (-114_{10}) \\
+ & 0 & 1 & 0 & \textcolor{red}{0} & 0 & 0 & 0 & 1 & (65_{10}) \\
\hline
= & 1 & 1 & 0 & \textcolor{blue}{0} & 1 & 1 & 1 & 1 & (-49)_{10}
\end{array}$$

Addition binaire entière signée

Pour additionner :

Pour additionner des entiers signés, on utilise le complément à 2 où le processus est le plus simple et unifié. La soustraction c'est l'addition de (-) la valeur.

L'addition est sont simple :

$$\begin{array}{rrrrrrrrr}
& 1 & 0 & \textcolor{red}{0} & 0 & 1 & 1 & 1 & 0 & (-114_{10}) \\
+ & 0 & 1 & \textcolor{red}{0} & 0 & 0 & 0 & 0 & 1 & (65_{10}) \\
\hline
= & 1 & 1 & \textcolor{blue}{0} & 0 & 1 & 1 & 1 & 1 & (-49)_{10}
\end{array}$$

Addition binaire entière signée

Pour additionner :

Pour additionner des entiers signés, on utilise le complément à 2 où le processus est le plus simple et unifié. La soustraction c'est l'addition de (-) la valeur.

L'addition est sont simple :

$$\begin{array}{rrrrrrrrrr}
& 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & (-114_{10}) \\
+ & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & (65_{10}) \\
\hline
= & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & (-49)_{10}
\end{array}$$

Addition binaire entière signée

Pour additionner :

Pour additionner des entiers signés, on utilise le complément à 2 où le processus est le plus simple et unifié. La soustraction c'est l'addition de (-) la valeur.

L'addition est simple :

$$\begin{array}{r}
 \quad \textcolor{red}{1} \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad (-114_{10}) \\
 + \quad \textcolor{red}{0} \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1 \quad (65_{10}) \\
 \hline
 = \quad \textcolor{blue}{1} \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \quad (-49)_{10}
 \end{array}$$

Attention

Le carry ou dépassement

Problème si le codage du résultat dépasse le nombre de bits possibles avec des opérations sur des non-signés. Peut être détecté avec 1 bit de sortie qui prend la dernière retenue.

$$\begin{array}{rcl}
 & 1011 & (11_{10}) \\
 + & 0111 & (7_{10}) \\
 \hline
 = & \textcolor{red}{1}0010 & (2_{10})
 \end{array}$$

Pour les signés, il y a carry si l'addition se fait entre deux nombres de même signe et si la dernière retenue est distincte du bit de poids fort. Exemple où pas de dépassement :

$$\begin{array}{rcl}
 & 1101 & (-3_{10}) \\
 + & 1110 & (-2_{10}) \\
 \hline
 = & \textcolor{red}{1}1011 & (-5_{10})
 \end{array}$$

Overflow

L'overflow ou débordement

Problème si le codage du résultat utilise le bit de poids fort pour la valeur.
Peut être détecté si les deux opérandes sont du même signe et le résultat de signe différent

$$\begin{array}{rcl} & 0011 & (3_{10}) \\ + & 0111 & (7_{10}) \\ \hline = & 1010 & (-6_{10}) \end{array}$$

Carry

Décimal	Codage binaire	Décimal	Codage binaire
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

Le carry fait retourner à 0 après 15 (et vice-versa). Exemple à noter

Overflow

Chaîne de bits	Complément à 2
0000	+0
0001	+1
0010	+2
0011	+3
0100	+4
0101	+5
0110	+6
0111	+7
1000	-8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

Le carry fait retourner à 0 après -1 (et vice-versa). Exemple à noter

Multiplication binaire entière

Rappel :

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

La multiplication :

	0	0	1	1	0	1	1	0	(54_{10})
$\times$	0	0	0	0	0	1	1	1	(7_{10})
	0	0	1	1	0	1	1	0	

Multiplication binaire entière

Rappel :

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

La multiplication :

		0	0	1	1	0	1	1	0	(54 ₁₀)
×		0	0	0	0	0	1	1	1	(7 ₁₀)
		0	0	1	1	0	1	1	0	
	0	0	1	1	0	1	1	0		

Multiplication binaire entière

Rappel :

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

La multiplication :

		0	0	1	1	0	1	1	0	(54 ₁₀)
×		0	0	0	0	0	1	1	1	(7 ₁₀)
		0	0	1	1	0	1	1	0	
	0	0	1	1	0	1	1	0		
	0	0	1	1	0	1	1	0		
	0	0	1	1	0	1	1	0		

Multiplication binaire entière

Rappel :

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

La multiplication :

			0	0	1	1	0	1	1	0	(54_{10})
×			0	0	0	0	0	1	1	1	(7_{10})
			0	0	1	1	0	1	1	0	
+		0	0	1	1	0	1	1	0		
+	0	0	1	1	0	1	1	0			
	0	1	0	1	1	1	1	0	1	0	(378_{10})

Multiplication binaire entière

Rappel :

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

La multiplication :

$$\begin{array}{r}
 \begin{array}{cccccccc}
 & & & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & (54_{10}) \\
 \times & & & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & (7_{10}) \\
 \hline
 & & & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & \\
 + & & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & \\
 + & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & & \\
 \hline
 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & (378_{10})
 \end{array}
 \end{array}$$

Toujours risque de carry et overflow

Division binaire entière

Division :

La division est obtenue par itération de soustractions jusqu'à ce que le résultat soit inférieur au diviseur. C'est la division euclidienne :

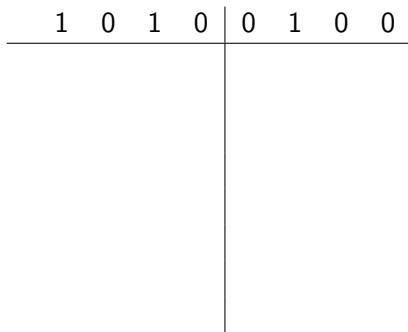
- Quotient = nombre de soustractions
- Reste = résultat de la dernière soustraction

Exemple :

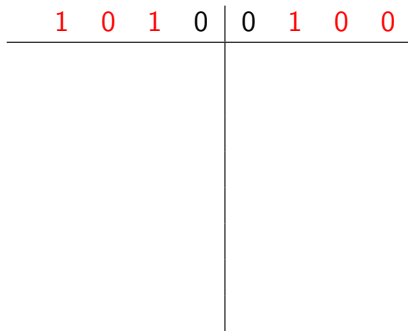
$$\begin{array}{r}
 00001010 \quad (10_{10}) \\
 - 00000100 \quad (4_{10}) \\
 \hline
 00000110 \quad (6_{10}) \\
 - 00000100 \quad (4_{10}) \\
 \hline
 00000010 \quad (2_{10})
 \end{array}$$

Quotient = 2 et Reste = 2

Autre méthode



Autre méthode



Autre méthode

$$\begin{array}{rcccc|cccc}
 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\
 \hline
 & 1 & 0 & 1 & & 1 & & & \\
 - & 1 & 0 & 0 & & & & & \\
 = & 0 & 0 & 1 & & & & &
 \end{array}$$

Autre méthode

$$\begin{array}{r}
 \begin{array}{cccc|cccc}
 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\
 \hline
 & & & & 1 & 0 & &
 \end{array} \\
 < \begin{array}{cccc|}
 & 0 & 1 & 0 & \\
 & 1 & 0 & 0 & \\
 & 0 & 1 & 0 &
 \end{array}
 \end{array}$$

Autre méthode

$$\begin{array}{r|rrrr}
 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\
 \hline
 & & & & 1 & 0 & & \\
 \\
 & 0 & 1 & 0 & & & &
 \end{array}$$

Quotient = $0010 = 2_{10}$ et Reste = $0010 = 2_{10}$

Addition binaire flottant

Addition :

Addition possible uniquement si les deux nombres ont le même exposant.
Si l'exposant est différent, on décale le nombre avec le plus petit exposant.

Exemple :

$$1.0011000 \times 2^2 + 1.1101011 \times 2^1 \rightarrow 1.0011000 \times 2^2 + 0.1110101\mathbf{1} \times 2^2$$

Addition binaire flottant

Addition :

Addition possible uniquement si les deux nombres ont le même exposant.
Si l'exposant est différent, on décale le nombre avec le plus petit exposant.

Exemple :

$$1.0011000 \times 2^2 + 1.1101011 \times 2^1 \rightarrow 1.0011000 \times 2^2 + 0.1110101 \times 2^2$$

$$4.75_{10} + 3.671875_{10} \rightarrow 4.75_{10} + 3.65625_{10}$$

Addition binaire flottant

Addition :

Addition possible uniquement si les deux nombres ont le même exposant.
Si l'exposant est différent, on décale le nombre avec le plus petit exposant.

Exemple :

$$1.0011000 \times 2^2 + 1.1101011 \times 2^1 \rightarrow 1.0011000 \times 2^2 + 0.1110101 \times 2^2$$

$$4.75_{10} + 3.671875_{10} \rightarrow 4.75_{10} + 3.65625_{10}$$

$$\begin{array}{r}
 1.0011000 \times 2^2 \quad (4.75_{10}) \\
 + \quad 0.1110101 \times 2^2 \quad (3.65625_{10}) \\
 \hline
 10.0001101 \times 2^2 \quad (8.40625_{10})
 \end{array}$$

Addition binaire flottant

Addition :

Addition possible uniquement si les deux nombres ont le même exposant.
Si l'exposant est différent, on décale le nombre avec le plus petit exposant.

Exemple :

$$1.0011000 \times 2^2 + 1.1101011 \times 2^1 \rightarrow 1.0011000 \times 2^2 + 0.1110101 \times 2^2$$

$$4.75_{10} + 3.671875_{10} \rightarrow 4.75_{10} + 3.65625_{10}$$

$$\begin{array}{r}
 1.0011000 \times 2^2 \quad (4.75_{10}) \\
 + \quad 0.1110101 \times 2^2 \quad (3.65625_{10}) \\
 \hline
 1.0000110 \times 2^3 \quad (8.375_{10})
 \end{array}$$

Addition binaire flottant

Addition :

Addition possible uniquement si les deux nombres ont le même exposant.
Si l'exposant est différent, on décale le nombre avec le plus petit exposant.

Exemple :

$$1.0011000 \times 2^2 + 1.1101011 \times 2^1 \rightarrow 1.0011000 \times 2^2 + 0.1110101 \times 2^2$$

$$4.75_{10} + 3.671875_{10} \rightarrow 4.75_{10} + 3.65625_{10}$$

$$\begin{array}{r}
 1.0011000 \times 2^2 \quad (4.75_{10}) \\
 + \quad 0.1110101 \times 2^2 \quad (3.65625_{10}) \\
 \hline
 1.0000110 \times 2^3 \quad (8.375_{10})
 \end{array}$$

Soustraction :

Même fonctionnement pour les soustractions

Multiplication binaire flottante

Multiplication :

Multiplication des mantisses et ajout des exposants

Exemple :

$$\begin{array}{rcl}
 & 1.0011001 & \times 2^2 \quad (4.78125_{10}) \\
 \times & 1.1100000 & \times 2^3 \quad (14_{10}) \\
 \hline
 & 1.001100100 & \\
 & 0.100110010 & \\
 & 0.010011001 & \\
 \hline
 \end{array}$$

Multiplication binaire flottante

Multiplication :

Multiplication des mantisses et ajout des exposants

Exemple :

$$\begin{array}{rcl}
 & 1.0011001 & \times 2^2 \quad (4.78125_{10}) \\
 \times & 1.1100000 & \times 2^3 \quad (14_{10}) \\
 \hline
 & 1.001100100 & \\
 & 0.100110010 & \\
 & 0.010011001 & \\
 \hline
 \end{array}$$

Multiplication binaire flottante

Multiplication :

Multiplication des mantisses et ajout des exposants

Exemple :

$$\begin{array}{rcl}
 & 1.0011001 & \times 2^2 \quad (4.78125_{10}) \\
 \times & 1.1100000 & \times 2^3 \quad (14_{10}) \\
 \hline
 & 1.001100100 & \\
 & 0.100110010 & \\
 & 0.010011001 & \\
 \hline
 & 10.000101111 & \times 2^5 \quad (66.9375_{10})
 \end{array}$$

Multiplication binaire flottante

Multiplication :

Multiplication des mantisses et ajout des exposants

Exemple :

$$\begin{array}{rcl}
 & 1.0011001 & \times 2^2 \quad (4.78125_{10}) \\
 \times & 1.1100000 & \times 2^3 \quad (14_{10}) \\
 \hline
 & 1.001100100 & \\
 & 0.100110010 & \\
 & 0.010011001 & \\
 \hline
 & 10.000101111 & \times 2^5 \quad (66.9375_{10})
 \end{array}$$

Multiplication binaire flottante

Multiplication :

Multiplication des mantisses et ajout des exposants

Exemple :

$$\begin{array}{rcl}
 & 1.0011001 & \times 2^2 \quad (4.78125_{10}) \\
 \times & 1.1100000 & \times 2^3 \quad (14_{10}) \\
 \hline
 & 1.001100100 & \\
 & 0.100110010 & \\
 & 0.010011001 & \\
 \hline
 & 1.0000101 & \times 2^6 \quad (66.5_{10})
 \end{array}$$

Division binaire flottante

Division :

Ne sera pas étudié dans ce cours. C'est une procédure plus compliquée.

Idées :

Division des mantisse et soustraction des exposants. Si quotient inférieur à 1 alors décaler d'une position à gauche. Le reste à le même signe que le dividende. La division des mantisses ne s'arrête que quand le reste est à 0.

Plan

- 1 Historique
- 2 Le codage de l'information
- 3 Arithmétique
- 4 Algèbre de Boole et circuits logiques**
- 5 Architecture général

Algèbre de bool

Rappel :

a	b	a and b	a or b	not a
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Petits ajouts :

- $A \wedge B$ ($A \times B$, AB , $A.B$ ou A and B)
- $A \vee B$ ($A + B$ ou A or B)
- $\neg A$ ($\overline{A}$ ou $not A$)
- ex : $\neg(a \wedge b) \vee c$
- $\neg(a \wedge b) = \neg a \vee \neg b$ (Loi de De Morgan)

Algèbre de Bool

Associativité de $\vee$	$(A + B) + C = A + (B + C)$
Associativité de $\wedge$	$(A \times B) \times C = A \times (B \times C)$
Commutativité de $\vee$	$A + B = B + A$
Commutativité de $\wedge$	$A \times B = B \times A$
Distributivité de $\wedge$ avec $\vee$	$A \times (B + C) = (A \times B) + (A \times C)$
Distributivité de $\vee$ avec $\wedge$	$A + (B \times C) = (A + B) \times (A + C)$
Identité $\vee$	$A + 0 = A$
Identité $\wedge$	$A \times 1 = A$
Élément absorbant $\vee$	$A + 1 = 1$
Élément absorbant $\wedge$	$A \times 0 = 0$
Idempotence $\vee$	$A + A = A$
Idempotence $\wedge$	$A \times A = A$
Absorption $\vee$	$A \times (A + B) = A$
Absorption $\wedge$	$A + (A \times B) = A$
Complémentarité $\vee$	$A + \neg A = 1$
Complémentarité $\wedge$	$A \times \neg A = 0$
Complémentarité $\neg$	$\neg \neg A = A$

Les tables de vérité

Les fonctions logiques :

Les ordinateurs numériques binaires expriment toutes les opérations comme fonctions logique.

$$f : \{0, 1\}^n \rightarrow \{0, 1\}$$

a_0	a_1	a_2	$f(a_0, a_1, a_2)$ Complète	$g(a_0, a_1, a_2)$ Incomplète
0	0	0	1	0
0	0	1	0	1
0	1	0	0	x
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	0	x
1	1	1	0	x

La forme canonique disjonctive

a_0	a_1	a_2	$f(a_0, a_1, a_2)$
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Forme canonique disjonctive :

Somme valant 1 des mintermes. Un minterme est regroupe toutes les variables booléennes liées par des $\wedge$.

$$f(a_0, a_1, a_2) =$$

La forme canonique disjonctive

a_0	a_1	a_2	$f(a_0, a_1, a_2)$
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Forme canonique disjonctive :

Somme valant 1 des mintermes. Un minterme est regroupe toutes les variables booléennes liées par des $\wedge$.

$$f(a_0, a_1, a_2) = \neg a_0 \neg a_1 \neg a_2$$

La forme canonique disjonctive

a_0	a_1	a_2	$f(a_0, a_1, a_2)$
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Forme canonique disjonctive :

Somme valant 1 des mintermes. Un minterme est regroupe toutes les variables booléennes liées par des $\wedge$.

$$f(a_0, a_1, a_2) = \neg a_0 \neg a_1 \neg a_2 + a_0 \neg a_1 \neg a_2$$

La forme canonique disjonctive

a_0	a_1	a_2	$f(a_0, a_1, a_2)$
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Forme canonique disjonctive :

Somme valant 1 des mintermes. Un minterme est regroupe toutes les variables booléennes liées par des $\wedge$.

$$f(a_0, a_1, a_2) = \neg a_0 \neg a_1 \neg a_2 + a_0 \neg a_1 \neg a_2 + a_0 \neg a_1 a_2$$

La forme canonique disjonctive

a_0	a_1	a_2	$f(a_0, a_1, a_2)$
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Forme canonique disjonctive :

Somme valant 1 des mintermes. Un minterme est regroupe toutes les variables booléennes liées par des $\wedge$.

$$f(a_0, a_1, a_2) = (\overline{a_0} \wedge \overline{a_1} \wedge \overline{a_2}) \vee (a_0 \wedge \overline{a_1} \wedge \overline{a_2}) \vee (a_0 \wedge \overline{a_1} \wedge a_2)$$

Le tableau de Karnaugh

Le tableau de Karnaugh :

Représenter l'état des variables sur les lignes et colonnes et on mets le résultat dans la case.

Remarque importante :

On utilise le code de Gray

Le tableau de Karnaugh

a_0	a_1	a_2	$f(a_0, a_1, a_2)$
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

=

$a_1 a_2$		00	01	11	10
a_0					
0		1	0	0	0
1		1	1	0	0

Le tableau de Karnaugh

		$a_1 a_2$			
		00	01	11	10
a_0	0	1	0	0	0
	1	1	1	0	0

La simplification :

Le tableau de Karnaugh

		$a_1 a_2$			
		00	01	11	10
a_0	0	1	0	0	0
	1	1	1	0	0

La simplification :

$$a_0 \neg a_1 \neg a_2 + a_0 \neg a_1 a_2 = a_0 \neg a_1$$

Le tableau de Karnaugh

		$a_1 a_2$			
		00	01	11	10
a_0	0	1	0	0	0
	1	1	1	0	0

La simplification :

$$a_0 \neg a_1 \neg a_2 + a_0 \neg a_1 a_2 = a_0 \neg a_2$$

$$a_0 \neg a_1 \neg a_2 + \neg a_0 \neg a_1 \neg a_2 = \neg a_1 \neg a_2$$

Le tableau de Karnaugh

		a_1a_2			
		00	01	11	10
a_0	0	1	0	0	0
	1	1	1	0	0

La simplification :

$$a_0 \neg a_1 \neg a_2 + a_0 \neg a_1 a_2 = a_0 \neg a_1$$

$$a_0 \neg a_1 \neg a_2 + \neg a_0 \neg a_1 \neg a_2 = \neg a_1 \neg a_2$$

$$f(a_0, a_1, a_2) = a_0 \neg a_1 + \neg a_1 \neg a_2$$

Le tableau de Karnaugh

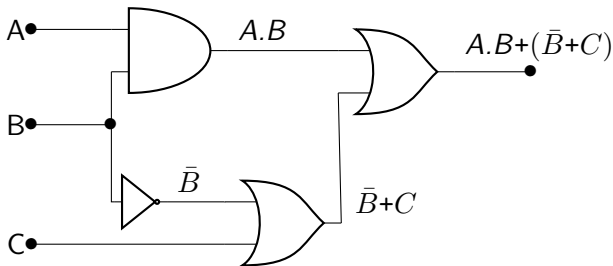
		a_2a_3			
		00	01	11	10
a_0a_1	00	1	0	0	1
	01	0	0	0	0
	11	0	0	0	0
	10	1	0	0	1

$$f(a_0, a_1, a_2, a_3) = \neg a_1 \neg a_3$$

Les portes et circuits logiques

Les portes logiques :

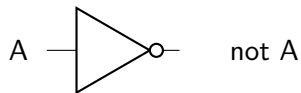
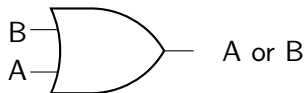
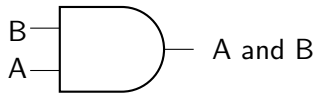
Expressions booléenne dans les circuits électroniques (c-à-d, les ordinateurs)



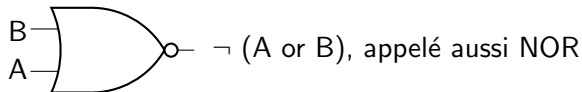
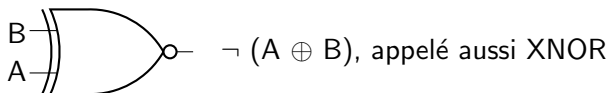
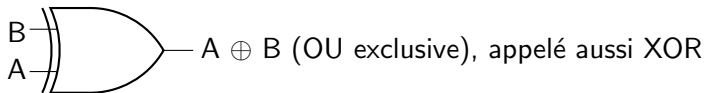
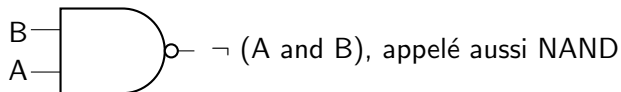
Courant de contrôle et courant contrôlé

Rappel : les portes logiques sont basées sur le principe de courant de contrôle et courant contrôlé

Les portes logiques



Les portes logiques



La complétude fonctionnelle

Ensemble fonctionnel complet :

Un ensemble de portes logiques qui peuvent représenter toutes les fonctions logiques

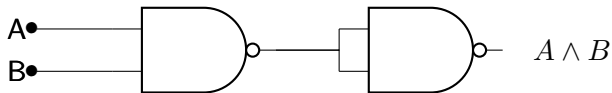
Ensemble minimal :

Un ensemble est minimal si on ne peut pas retirer d'élément sans perdre la complétude fonctionnelle

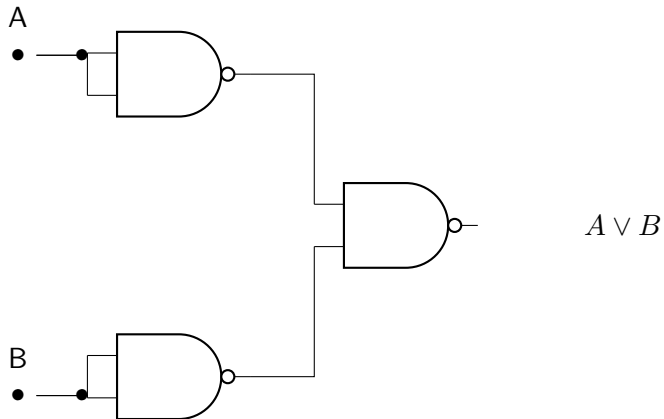
Exemple :

- $\{\text{ET}, \text{OU}, \text{NOT}, \text{NAND}, \text{NOR}\}$ est fonctionnel complet mais pas minimal
- $\{\text{ET}, \text{NOT}\}$ est fonctionnel complet et minimal
- $\{\text{NAND}\}$ est fonctionnel complet et minimal
- $\{\text{NOR}\}$ est fonctionnel complet et minimal

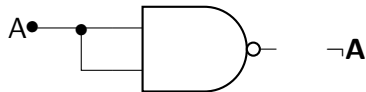
NAND vers AND



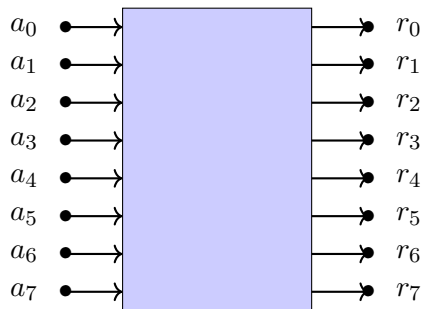
NAND vers OR



NAND vers NOT



Circuit combinatoire



Circuit combinatoire :

Une “boîte” qui prend un ensemble de bits en entrée et avec un ensemble de bits en sortie. Les bits de la sortie sont en **fonction** des bits entrées

Circuit combinatoire

Construire un circuit combinatoire

- Produire les tables de vérités pour chaque bit
- Produire la forme canonique
- Simplifier l'expression (table de Karnaugh)
- Créer le résultat en portes logiques

Le demi-additionneur 1 bit

Le demi-additionneur :

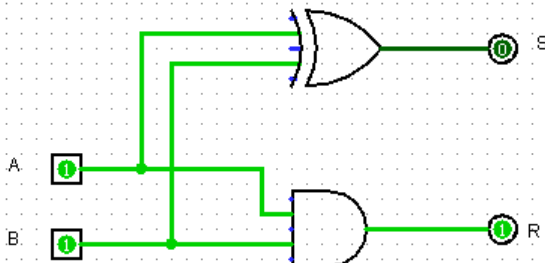
Addition de deux bits, A et B. Produit un bit de retenue R et un bit de somme S.

+	0	1
0	00	01
1	01	10

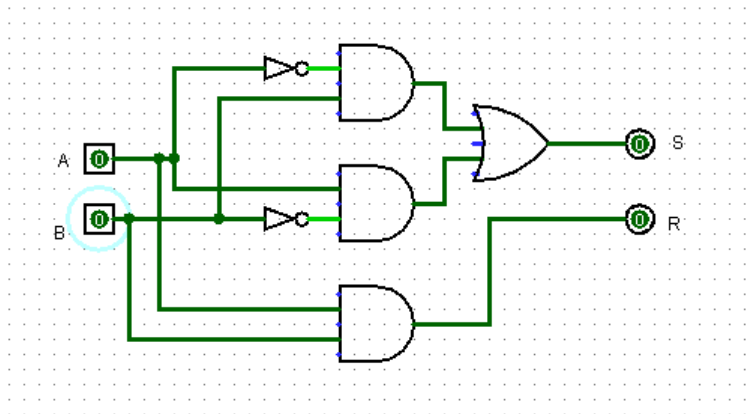
=

A	B	R	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Le circuit logique – Demi-additionneur



Le circuit logique (forme canonique) – Demi-additionneur



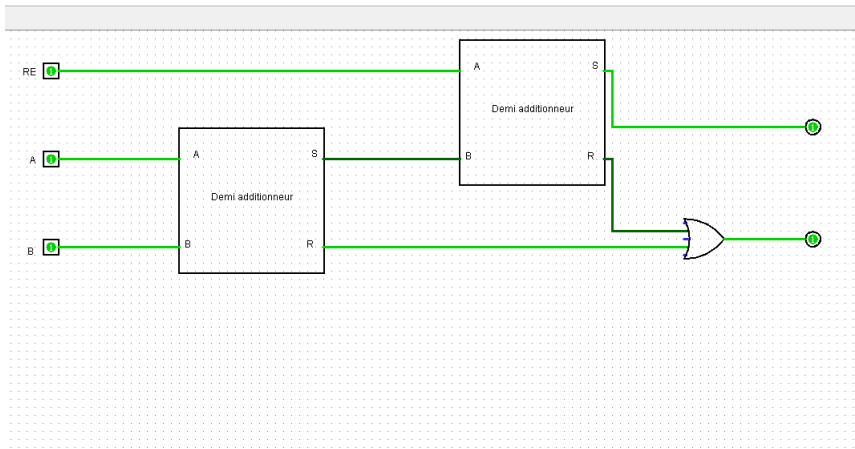
L'additionneur 1 bit

L'additionneur :

Addition avec trois bits, A et B et une retenue RE (de l'addition des bits précédents). Produit un bit de retenue RS et un bit de somme S.

A	B	RE	S	RS
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

le circuit logique – Additionneur



Et pour une addition à n bits ?

Et pour une addition à n bits ? On répète en cascade l'addition
(exercice de TP)

Le décodeur

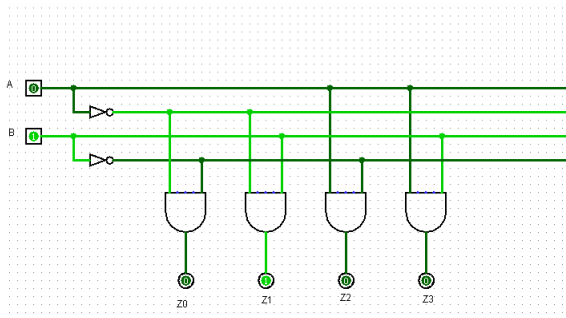
Décodeur

Associer chaque ensemble de valeurs possibles en entrée à un bit de sortie différent

Une seule sortie à 1 parmi 2^n bits de sortie (n bits d'entrée)

A	B	Z0	Z1	Z2	Z3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

le circuit logique – Décodeur



Le multiplexeur/démultiplexeur

Multiplexeur :

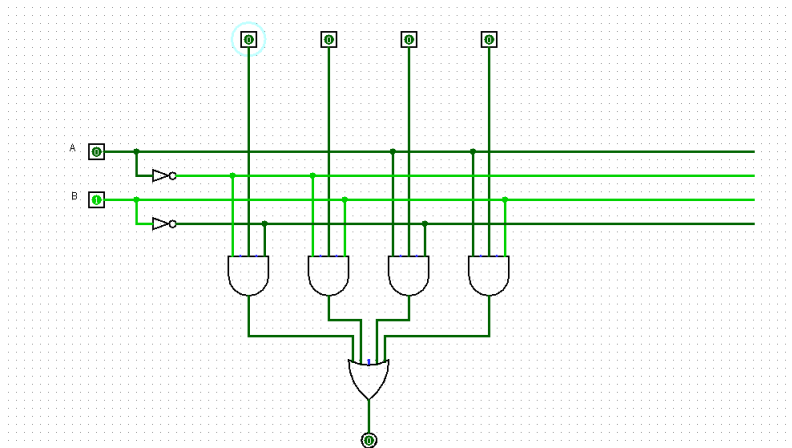
2^n bits de data, n bits de sélection et une sortie. Permet d'aiguillonner (ou sélectionner) l'un des 2^n d'entrée vers la sortie.

Démultiplexeur :

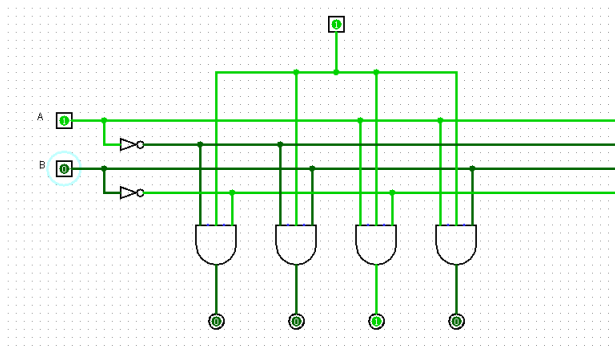
1 bit de data, n bits de sélection et 2^n sorties. Permet d'aiguillonner (ou sélectionner) le bit d'entrée vers une des sorties.

A	B	Bit sélectionné
0	0	Z0
0	1	Z1
1	0	Z2
1	1	Z3

le circuit logique – Multiplexeur



le circuit logique – Démultiplexeur



Les circuits séquentiels

Circuit combinatoire :

Les sorties sont modifiées dès que les entrées changent (vue jusqu'à maintenant).

Circuit séquentiel :

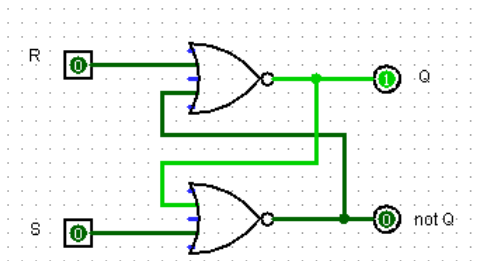
Possibilités de mémorisation, et la valeur des sorties à un instant t dépend des entrées en t et des sorties en $t - 1$.

Bascule RS :

Un bit pour représenter la sortie à 0, et un bit pour la sortie à 1. Deux bits (reset et set) qui permettent de changer la valeur.

Circuit logique – Bascule RS

S (set)	R (reset)	Q' (Bit mémorisé)	Commentaire
0	0	Q	Q' conserve son état
1	0	1	Q' est set à 1
0	1	0	Q' est reset à 0
1	1	x	Non autorisé (Q et $\neg Q$ à 0)



Synchrone et Asynchrone

Bascule RS :

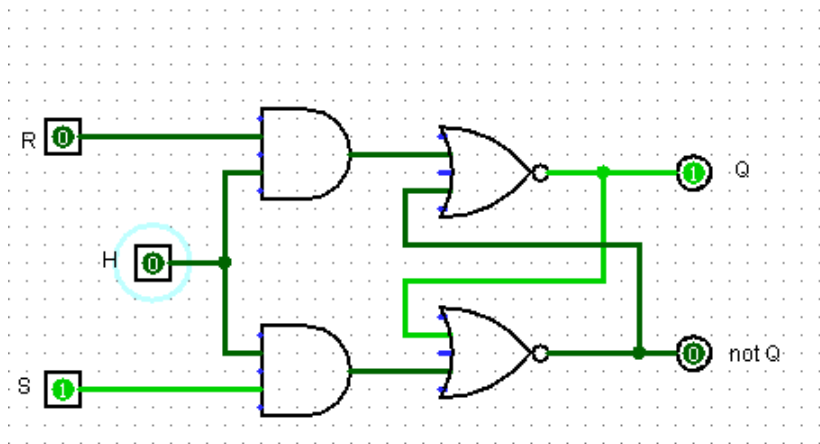
Bascule asynchrone, les sorties (Q et $\overline{Q}$) sont modifiées dès que les entrées changent. Problèmes :

- Sensibilité aux bruits parasites
- Synchronisation (quand peut on utiliser les sorties ?). Dépend de la vitesse de commutation

Bascule RSH:

Bascule synchrone. La sortie est dite synchrone (à l'horloge H) car seul un évènement sur l'horloge peut faire évoluer sa sortie.

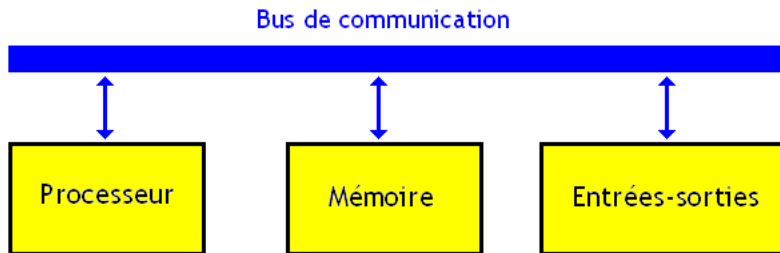
Circuit logique – Bascule RSH



Plan

- 1 Historique
- 2 Le codage de l'information
- 3 Arithmétique
- 4 Algèbre de Boole et circuits logiques
- 5 Architecture général**

L'architecture simplifiée



La mémoire

La mémoire c'est un tableau de cases avec une adresse pour chaque case

	⋮	⋮
4	10111011	
3	00111110	
2	01100000	
1	11001001	
0	11111101	

Mémoires

Format

- Bit : 0 ou 1
- Octet (*bytes*) : 8 bits
- Mot : Unité d'information adressable (32 ou 64 bits typiquement)

Caractéristique

- Capacité : nombre d'octets
- Temps d'accès : temps pour effectuer une lecture ou une écriture
- Cycle mémoire : temps minimum entre deux accès
- Débit : nombre d'octets lus par seconde
- Volatilité : perte ou non du contenu quand l'ordinateur est éteint

Mémoires

ROM

- Read Only Memory, aussi appelé mémoire morte
- Mémoire non-volatile (ne s'efface pas quand pas d'électricité)
- Ne pouvait originellement pas être modifiée
- Typiquement stock des programmes ou micro-programmes (ex: BIOS)

RWM

- Read Write Memory
- Peut être accédée en lecture et écriture
- Random Access Memory (RAM), mémoire vive et volatile
- Mémoire dure
- Mémoire de travail (variables d'un programme)

Performances des mémoires

Principe de localité

Un programme passe 90% du temps dans 10% du code

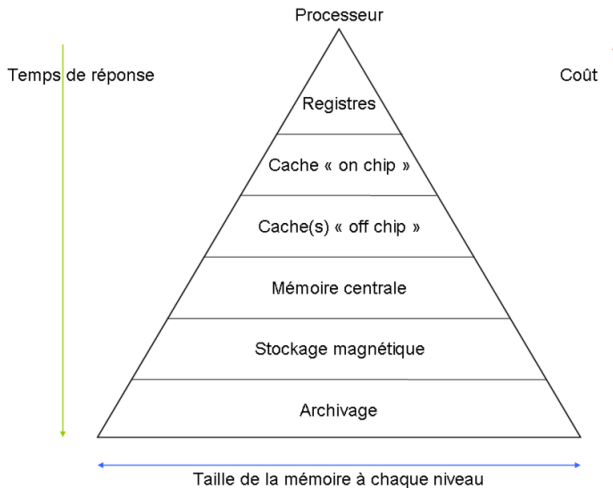
Localité temporelle

Une instruction ou une donnée utilisée récemment va probablement être réutilisée bientôt

Localité spatiale

Un programme à tendance à utiliser des instructions et des données consécutives qui sont proches dans la mémoire

Performances des mémoires



Les registres

Les registres:

- Utilisés par le processeur pour stocker des instructions ou des valeurs nécessaire à ses calculs
- Registres architecturaux (utilisés lors de la programmation du langage machine) et les registres internes (automatiquement utilisés par le processeur)
- Limités par la place disponible sur le microprocesseur
- Registres pour les instructions, des valeurs temporaires,...
- 10-100 registres de ≈ 32 ou 64 bits
- Temps d'accès en 1 nano-seconde

Les caches

Les caches

Conservent des données accédées récemment proches du processeur, typiquement pour le principe de localité. Exemple : instructions des boucles. Évite ainsi un accès plus long à la mémoire RAM. 10 à 100 nano-secondes en temps d'accès.

Le niveau L1

Le cache interne au processeur. De 32 à 256 Ko environ.

Le niveau L2

Le cache interne ou externe au processeur. De 128 Ko à 24 Mo environ.

Mémoire centrale

Mémoire centrale

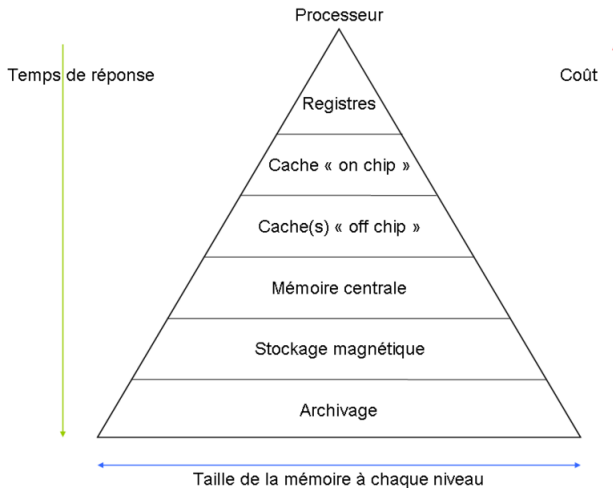
- Random Access Memory (et pas Memories, ça c'est les Daft Punk)
- Accès direct (utilise des adresses)
- Volatile donc effacé quand l'ordinateur est éteint
- Maintenir des résultats de calculs, contenir temporairement des programmes à exécuter, etc
- 4-32 Go environ
- 1 micro-seconde de temps d'accès

Mémoire de masse

Mémoire de masse

- Stocke de manière permanente des données
- Non-volatile
- Disque dur, clé USB, CD, . . .
- Plusieurs To
- 1 milli-seconde en temps d'accès

Performances des mémoires



Processeur

Le processeur

Unité de traitement (Central Processing Unit, CPU) c'est le cerveau de l'ordinateur : il effectue les calculs

Le projet se focalise sur la création d'un ordinateur

Processeur

Le jeu d'instructions

Chaque processeur possède son propre jeu d'instructions qui représente les calculs/actions que peut faire le processeur. Tout ce que fait l'ordinateur, ce n'est que répéter ces instructions. Les instructions dépendent de l'architecture du processeur (des sous-circuits logiques qui le composent)

Les instructions

- Les instructions arithmétiques
- Les instructions de test
- Les instructions d'accès mémoire
- Les instructions de saut (pour les while et if)
- ... etc. ...

Architecture du processeur

L'architecture externe/interne

- Niveau 2 (externe) : le langage machine (jeu d'instructions + registres disponibles)
- Niveau 1 (interne) : l'implémentation des instructions (diagramme des composantes du processeur, de leurs connections et de leurs interactions)
- Niveau 0 : réalisation en physique et logique numérique

Les jeux d'instructions

- Processeur à jeu d'instructions réduit (RISC)
- Processeur à jeu d'instructions étendu (CISC)

Langage machine

Langage machine

Le langage machine est le langage du processeur: ce sont des expressions en bits qui codent les opérations et les adresses des valeurs utilisées.

Bas niveau

Le langage machine est le langage le plus bas niveau, c'est-à-dire que l'on manipule directement le processeur (registres, jeu d'instructions, . . .)

Langage machine

Une instruction

Un ensemble de bits divisé en une partie pour coder l'opération (**opcode**) et une partie pour les opérandes (ex : adresses des registres)

opcode	opérandes
a1	bf af 00

Illisible : 0a1158bcc885412379f9b8c42d548e245f67af...

L'assembleur

L'assembleur représente le langage machine de manière lisible pour l'être humain. On considère qu'un code (ex: en C++) est traduit en assembleur lors de la compilation.

- C++ : $x = a + b$;
- Assembleur : `add $v0, $t0, $t1`

- Langage machine :

ADD	\$v0	\$t0	\$t1	shamt	funct
100000	00010	01000	01001	00000	000000

Le processeur MIPS

- Architecture de Processeur de la fin des années 90
- Typique d'une machine RISC (peu d'instruction)
- Architecture populaire et présente dans de nombreux systèmes
- Beaucoup dans les systèmes embarqués
- Ex : PS1, PS2, N64, PSP, Routeurs, calculateur,...
- 57 instructions

Pour aller plus loin :

MIPS (architecture processeur) et MARS permettant de coder sur MIPS

Le processeur MIPS

Les registres Le processeur MIPS possède 32 registres de 32 bits et un registre de 32 bits pour le **PC** (program counter)

\$zero	Toujours égal à 0
\$at	Réservé pour l'assembleur
\$v0 et \$v1	Stockent les résultats des fonctions
\$a0 à \$a3	Stockent les 4 premiers paramètres d'une fonction
\$t0 à \$t7	Stockent des valeurs temporaires
\$s0 à \$s7	Sauvegardés et utilisés plus tard
\$t8 et \$t9	Stocke des valeurs temporaires
\$k0 et \$k1	Réservés au système
\$gp	Global pointer
\$sp	Stack pointer
\$fp	Frame pointer
\$ra	Adresse de retour

Format d'instructions

Plusieurs formats pour les instructions :

- Format **I**ntermediate pour les UALs :
opcode Registre1 Registre2 Valeur
- Format **J**ump pour les ruptures de séquence, c.-à.-d. les sauts :
opcode Valeur
- Format **R**egister pour les UALs :
opcode Registre1 Registre2 Registre3

L'assembleur

C++

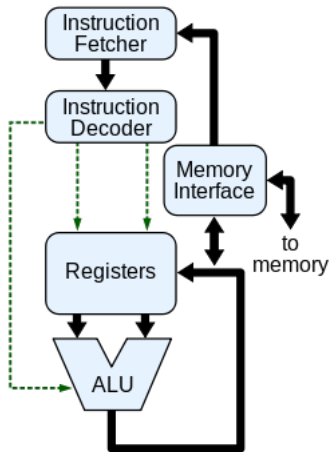
```
x = a + b;
y = c + d;
if (x > y) {
    return x;
} else {
    return y;
}
```

Assembleur

```
add $t0, $a0, $a1    # x (t0) = a (a0) + b (a1)
add $t1, $a2, $a3    # y (t1) = a (a2) + b (a3)
ble $t0, $t1, f.false # if x <= y, alors f.false
move $v0, $t0        # return x
```

```
f.false:
    move $v0, $t1    # return x
```

Diagramme simple processeur



Processeur

Les 5 étapes d'une instruction

- **Instruction Fetch** : charge l'instruction à effectuer
- **Instruction Decode** : décode l'instruction et adresse les registres
- **Execute** : exécute l'instruction (avec le registre) dans les UALs
- **Memory** : interaction avec la mémoire en lecture ou écriture
- **Write Back** : écrit dans le registre le résultat de l'instruction

A noter qu'un processeur effectue les calculs de tous les programmes, qui ne sont pas exécutés en parallèle mais en série

Pipeline

Le principe de pipeline :

Plusieurs instructions en simultané : une dans **Instruction Fetch**, une dans **Instruction Decode**, une dans **Execute**,...

Problèmes :

- Instruction dépendante du résultat d'une autre instruction
- Instruction dépendante du résultat d'un branchement
- Problème matériel (ex : écriture et lecture en même temps)

Solutions :

- Attente de quelques étapes
- Utilisation des données dès que disponible (ex : redirection du résultat de l'UAL avant l'étape **Memory** et **Write Back**)

Les performances

Comment comparer 2 systèmes ? 2 processeurs ? 2 programmes ?

Les performances

Performance sur le temps

$$\text{Performance} = \frac{1}{\text{temps d'exécution}}$$

Quels temps d'exécution ?

- Wall-clock time (Elapsed real time) : temps passé à attendre le résultat du programme
- CPU time c'est le temps de travail du CPU
- Le temps d'accès à la mémoire
- User time c'est le temps passé sur le code de l'utilisateur
- System time est le temps passé en appel système
- Idle time est le temps d'attente (que le CPU se libère par exemple)

Temps CPU

Les cycles

Un cycle pour un processeur (ou période P) c'est :

$$P = \frac{1}{\text{Fréquence de l'horloge (Hz)}}$$

Une instruction

Une instruction traitée par le CPU passe par plusieurs étapes (sous-instructions), dépendant de l'architecture du processeur. Chaque étape nécessite un cycle, donc on calcule le nombre de cycle par instruction (CPI)

Le temps CPU

$$\text{Temps CPU} = \frac{\# \text{instructions} \times \text{CPI}}{\text{Fréquence de l'horloge (Hz)}}$$

IPS et FLOPS

Instructions Per Second (IPS)

$$\text{IPS} = \frac{\# \text{instructions}}{\text{temps d'exécution}}$$

Surtout efficace avec des ordinateurs avec une architecture similaire car sinon ce n'est pas forcément les mêmes opérations faites.

FLoating-Point instructions per Second (FLOPS)

$$\text{FLOPS} = \frac{\# \text{instructions flottantes}}{\text{temps d'exécution}} = \# \text{cœurs} \times \text{fréquence} \times \frac{FLOP}{\# \text{cycle}}$$

Utilise la plupart du temps le FPU (floating point unit), destinée uniquement aux calculs flottant à l'intérieur du CPU

Historique FLOPS

Chez les super-ordinateur :

- 1964 : 10^6 FLOPS
- 1985 : 10^9 FLOPS
- 1997 : 10^{12} FLOPS
- 2008 : 10^{15} FLOPS
- Actuellement (2021) : 400×10^{15} FLOPS
- Bientôt ($\approx$ 2021) : 10^{18} FLOPS

Ordinateurs pour particuliers :

- 2020 : 10^{12} FLOPS pour les consoles de salons

Ordinateur personnel : composantes

Ordinateur personnel : composantes

La carte mère

C'est le "socle" où l'on branche tous les éléments (mémoire, processeur, ...). Peut contenir quelques éléments simples comme une carte son ou une carte réseau. La carte mère est fixée au boîtier qui contiendra l'ordinateur. Contient aussi les différentes connectiques (ex : port USB).

Ordinateur personnel : composantes

La carte mère

C'est le "socle" où l'on branche tous les éléments (mémoire, processeur, ...). Peut contenir quelques éléments simples comme une carte son ou une carte réseau. La carte mère est fixée au boîtier qui contiendra l'ordinateur. Contient aussi les différentes connectiques (ex : port USB).

Le processeur

Il effectue les calculs et est installé directement sur la carte mère.

Ordinateur personnel : composantes

La carte mère

C'est le "socle" où l'on branche tous les éléments (mémoire, processeur, ...). Peut contenir quelques éléments simples comme une carte son ou une carte réseau. La carte mère est fixée au boîtier qui contiendra l'ordinateur. Contient aussi les différentes connectiques (ex : port USB).

Le processeur

Il effectue les calculs et est installé directement sur la carte mère.

La RAM

La RAM est la mémoire vive. Elle prend la forme de barrettes de mémoire que l'on installe directement sur la carte mère.

Ordinateur personnel : composantes

Ordinateur personnel : composantes

La carte graphique

La carte graphique, ou GPU, s'occupe des affichages de l'ordinateur. Il s'agit d'un processeur spécialisé pour l'affichage. Des connectiques permettent de brancher les écrans.

Ordinateur personnel : composantes

La carte graphique

La carte graphique, ou GPU, s'occupe des affichages de l'ordinateur. Il s'agit d'un processeur spécialisé pour l'affichage. Des connectiques permettent de brancher les écrans.

Le disque dur

Le disque dur (ou les disques durs) est la mémoire dure de l'ordinateur.

Ordinateur personnel : composantes

La carte graphique

La carte graphique, ou GPU, s'occupe des affichages de l'ordinateur. Il s'agit d'un processeur spécialisé pour l'affichage. Des connectiques permettent de brancher les écrans.

Le disque dur

Le disque dur (ou les disques durs) est la mémoire dure de l'ordinateur.

L'alimentation

L'alimentation s'occupe d'envoyer le courant dans les différentes composantes de l'ordinateur.

Ordinateur personnel : composantes

Ordinateur personnel : composantes

Les ventilateurs (et systèmes de refroidissement liquide)

Il y a beaucoup de ventilateurs dans un ordinateur: pour le processeur, pour la carte graphique, pour l'ensemble, pour l'alimentation.

Ordinateur personnel : composantes

Les ventilateurs (et systèmes de refroidissement liquide)

Il y a beaucoup de ventilateurs dans un ordinateur: pour le processeur, pour la carte graphique, pour l'ensemble, pour l'alimentation.

Le boîtier

Le boîtier est une boîte de plastique où l'on mets les composantes pour former notre ordinateur. Le boîtier n'est pas seulement un cube simple mais est souvent réfléchi pour permettre une bonne évacuation de la chaleur.

Ordinateur personnel : composantes

Les ventilateurs (et systèmes de refroidissement liquide)

Il y a beaucoup de ventilateurs dans un ordinateur: pour le processeur, pour la carte graphique, pour l'ensemble, pour l'alimentation.

Le boîtier

Le boîtier est une boîte de plastique où l'on mets les composantes pour former notre ordinateur. Le boîtier n'est pas seulement un cube simple mais est souvent réfléchi pour permettre une bonne évacuation de la chaleur.

Les entrées et sorties diverses

Le lecteur CD-ROM, DVD, Blu-Ray. La souris, le clavier, les écrans, ...

Système d'exploitation

Système d'exploitation

Un système d'exploitation (Operating system ou **OS** en anglais) est un programme (ou ensemble de programmes) qui offre une abstraction du matériel pour tous les logiciels. C'est l'**OS** qui gère la gestion des ressources de l'ordinateur (mémoire, accès CPU,...), les périphériques, l'exécution des programmes,....

- Gestion des processus : qui est exécuté sur le CPU ? en attente ?...
- Appel système : faire un affichage, obtenir la date,...
- Gestion de la mémoire : chargement des programmes en mémoire, utilisation de la mémoire vive,...
- Gestion des entrées et sorties : utilise des pilotes pour l'interaction entre E/S et ordinateur
- Gestion des fichiers : organisation de la mémoire dure